

MIO: Multiverse Debugging in the Face of Input/Output

TOM LAUWAERTS, Universiteit Gent, Belgium
MAARTEN STEEVENS, Universiteit Gent, Belgium
CHRISTOPHE SCHOLLIERS, Universiteit Gent, Belgium

Debugging non-deterministic programs on microcontrollers is notoriously challenging, especially when bugs manifest in unpredictable, input-dependent execution paths. A recent approach, called multiverse debugging, makes it easier to debug non-deterministic programs by allowing programmers to explore all potential execution paths. Current multiverse debuggers enable both forward and backward traversal of program paths, and some facilitate jumping to any previously visited states, potentially branching into alternative execution paths within the state space.

Unfortunately, debugging programs that involve input/output operations using existing multiverse debuggers can reveal inaccessible program states, i.e. states which are not encountered during regular execution. This can significantly hinder the debugging process, as the programmer may spend substantial time exploring and examining inaccessible program states, or worse, may mistakenly assume a bug is present in the code, when in fact, the issue is caused by the debugger.

This paper presents a novel approach to multiverse debugging, which can accommodate a broad spectrum of input/output operations. We provide the semantics of our approach and prove the correctness of our debugger, ensuring that despite having support for a wide range of input/output operations the debugger will only explore those program states which can be reached during regular execution.

We have developed a prototype, called MIO, leveraging the WARDuino WebAssembly virtual machine to demonstrate the feasibility and efficiency of our techniques. As a demonstration of the approach we highlight a color dial built with a Lego Mindstorms motor, and color sensor, providing a tangible example of how our approach enables multiverse debugging for programs running on an STM32 microcontroller.

CCS Concepts: • **Computer systems organization** → *Embedded software*; • **Software and its engineering** → **Semantics**; **Software testing and debugging**; *Integrated and visual development environments.*

Additional Key Words and Phrases: Multiverse Debugging, Embedded devices, WebAssembly

ACM Reference Format:
Tom Lauwaerts, Maarten Steevens, and Christophe Scholliers. 2025. MIO: Multiverse Debugging in the Face of Input/Output. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 358 (October 2025), 30 pages. https://doi.org/10.1145/3763136

1 Introduction

Debugging non-deterministic programs is a challenging task, since bugs may only appear in very specific execution paths [30, 67]. This is especially true for microcontroller programs, which typically interact heavily with the environment. This makes reproducing bugs unreliable and time-consuming, a problem that traditional debuggers do not account for. By contrast, multiverse debugging [92] is a novel technique that solves this problem by allowing programmers to explore all

Authors' Contact Information: Tom Lauwaerts, Universiteit Gent, Ghent, Belgium, Tom.Lauwaerts@UGent.be; Maarten Steevens, Universiteit Gent, Ghent, Belgium, Maarten.Steevens@UGent.be; Christophe Scholliers, Universiteit Gent, Ghent, Belgium, Christophe.Scholliers@UGent.be.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
© 2025 Copyright held by the owner/author(s).
ACM 2475-1421/2025/10-ART358
https://doi.org/10.1145/3763136

possible execution paths. A multiverse debugger allows users to move from one execution path to another, even jumping to arbitrary program states in parallel execution paths. This entails traveling both forwards and backwards in time, i.e. a multiverse debugger is also a time travel debugger. So far, existing implementations work on abstract execution models to explore all execution paths of a program [70–72, 92]. Within these semantics, only the internal state of the program is controlled.

Unfortunately, debugging programs that involve input/output (I/O) operations using existing multiverse debuggers can reveal inaccessible program states that are not encountered during regular execution. This is known as the probe effect [25], and can occur in multiverse debuggers when they do not account for the effect of I/O operations on the external environment when changing the program state. Encountering such states during the debugging session can significantly hinder the debugging process, as the programmer may mistakenly assume a bug is present in the code, when in fact, the issue is caused by the debugger. In this paper, we investigate how we can scale multiverse debugging to programs running on a microcontroller which interacts with the environment through I/O operations. This introduces three new challenges.

First, the effect of output operations on the environment can influence later states in the execution path, for example when a robot drives forward. Therefore, when stepping backwards, the changes made to the environment by the program must be reverted to stay consistent with normal execution. Otherwise, the debugger may enter inaccessible execution paths. This far-reaching probe effect, is especially difficult to control when making arbitrary jumps in the execution tree. Second, input operations from the external environment make it difficult to maintain reproducibility [24]. There are often too many possible execution paths to explore due to an infinite range of inputs. Furthermore, it is impractical for developers to enumerate all possible inputs, or to perfectly configure the environment to achieve a specific execution path every time. Without a way to handle the large ranges of possible inputs, the reproducibility of non-deterministic bugs in the multiverse debugger becomes challenging. Third, due to the hardware limitations of the microcontrollers that we target it is unfeasible to run the multiverse debugger entirely on the microcontroller. We thus need to expand multiverse debugging so that it can be used even in such a restricted environment.

1.1 Overview

In this paper, we present a new approach to multiverse debugging tackling the three main challenges posed in the introduction making several key scientific contributions.

First, we establish a well-defined set of I/O operations that are *deterministically reversible*, which is a crucial requirement for the correctness proof of our multiverse debugger. Second, we formalize our debugger as a *remote debugger* with *sparse snapshotting* semantics to accommodate the hardware limitations of microcontrollers. Third, we define our semantics such that the multiverse debugger can instrument input operations, which is needed to allow the programmer to interactively explore the execution tree. Finally, we formulate our semantics in a *general and abstract* way with *novel correctness criteria*, providing valuable guidelines to formalize debuggers in general.

To demonstrate the practical applicability of our formal semantics we present MIO, a prototype multiverse debugger build on top of the WARDuino WebAssembly virtual machine [54]. MIO can debug code on various microcontrollers, including the ESP32, STM32, and Raspberry Pi Pico.

Our implementation also makes several technical contributions. It is, to our knowledge, the first *concrete multiverse debugger* to operate on a real-world, multi-language ecosystem (i.e., WebAssembly), moving beyond existing multiverse debuggers which have primarily focused on *abstract models* [70–72, 92] Furthermore, MIO is a *remote* multiverse debugger specifically tailored for resource-constrained microcontrollers. To address the significant performance cost of state capture on these devices, this work implements and benchmarks for the first time a *sparse snapshotting*

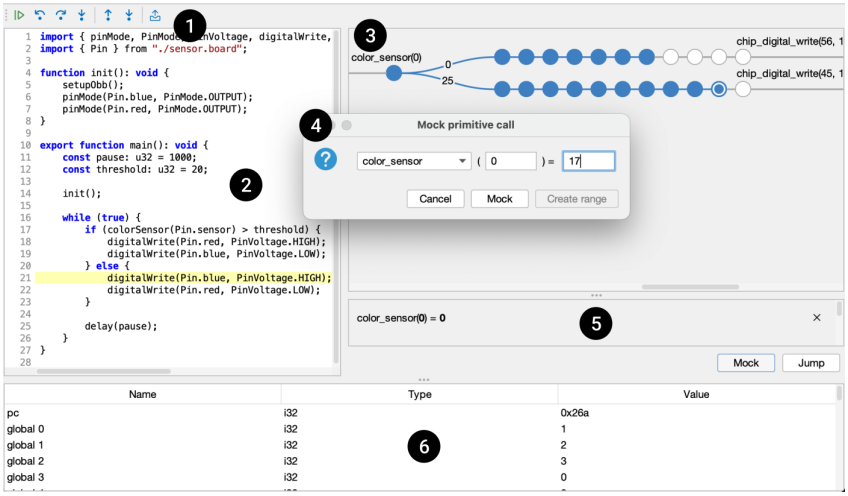


Fig. 1. Screenshot of the MIO debugger debugging a small light sensor program. Top (1): the debug operations; pause or continue, step back, step over, step into, step to the previous line, step to the next line, and update software. Left pane (2): source code. Top-right pane (3): the multiverse tree. Popup window (4): window to add mocked input. Bottom-right pane (5): the current mocked input. Bottom pane (6): general debugging information, such as the global and local variables, and the current program counter.

approach. Our results confirm that this technique is essential for achieving practical performance, mitigating the high overhead of traditional (full) snapshotting.

We start the article with an overview of multiverse debugging with I/O in Section 2, discussing the different parts of our prototype and the underlying challenges they overcome. This is followed by Section 3 with a detailed discussion of our formalism. We start this section with a discussion of the requirements for the I/O operations, and a brief summary of the key takeaways of the formalism. Next, in Section 4 we discuss our prototype implementation and show how the *deterministically reversible* primitives are implemented. In Section 5 we provide a quantitative evaluation of our snapshotting approach and discuss a case study showing how the technique works in practice. We conclude the article with the related work and conclusion in sections 6 and 7.

2 MIO: Multiverse Input/Output Debugger in Practice

Before we discuss the details of our contributions and the implementation, we give an overview of how our multiverse debugger, MIO, works in practice. We will use a simple example to illustrate the different concepts of our multiverse debugger.

2.1 Example: A Light Sensor Program

Consider a simple program that reads a value from a color sensor, to measure the light intensity, and turns on an LED with a color corresponding to the value read. The left side of Fig. 1 (2) shows the example program written in AssemblyScript [7]. Reading the value from the color sensor happens through the *colorSensor* function, and changing the color of the LED happens through the *digitalWrite* function. In an infinite loop, the program reads a value from the color sensor. If the value is below a threshold, the red LED is turned on, and otherwise the blue LED is turned on. At the end of the loop the program waits for 1000 milliseconds before starting the next iteration.

2.2 The Frontend of the Multiverse Debugger

Figure 1 shows the MIO multiverse debugger in action. While debugging, the program and the debugger backend run on a microcontroller connected to a computer running the debugger frontend, shown in the screenshot. The left pane (2) shows the program being debugged, in this case the light sensor program. In the top left corner (1), the user can see the debug operations available, which are in order; pause or continue, step back, step over, step into, step to the previous line, step to the next line, and finally, update the code on the microcontroller. The top-right pane (3) shows the multiverse tree, where each edge represents a WebAssembly instruction. In MIO all nodes are considered unique states, which means that loops are unrolled and no states will ever collapse into each other, making this graph always a rooted tree. MIO allows input values to be mocked using the *Mock* button, which triggers a popup window (4) where users can specify the mocked return value for a given input primitive. The *Create range* button can be used to add a range of new branches to the multiverse tree. The mocked primitives show up in the bottom-right pane (5), where they can also be removed again. The bottom pane (6) shows the global and local WebAssembly variables, and other program state information.

While debugging the light sensor program, the user can pause the program at any moment. At this point, the right pane will show the already explored paths of the multiverse tree. It is important to note that the multiverse tree does not correspond to a control flow graph, but shows every succeeding program state. Every edge in the tree represents the concrete execution of a single WebAssembly instruction, and every node represents the program state after that instruction. The multiverse tree in Fig. 1 labels every node before the execution of a primitive with the primitive name, and labels the outgoing edges with the associated return value.

2.3 Debugging with the Multiverse Debugger

A developer can use the multiverse debugger to explore the execution of the light sensor program, using the debug operations in the top left corner, following normal debugging conventions. While the right panel continually updates to show where in the program's execution the user is. However, the multiverse debugger allows for more than just stepping forwards and backwards through the program. The user can also jump to any node in the multiverse tree by simply clicking on that node, and explore the program from that point onwards with the available debugging instructions.

2.4 Exploring the Multiverse Tree

The screenshot in Fig. 1 shows the program paused at line 21, after reading a value from the color sensor. This value was 0, and if the user steps forward, the program will turn on the blue LED (pin 56) as shown by the multiverse tree. In an earlier stage, the user actually read a value of 25. Using the debugger, it is possible to explore that execution path again, without having to recreate the situation where the sensor reads 25. This is done by clicking on any node on the execution path where the sensor reads 25, which is shown as a blue node with a white circle in it. If the user presses the *Jump* button, the debugger will traverse the blue-indicated path in the multiverse tree, from the current node to the selected node. When reading the sensor value, the debugger will use mock input actions, to override the normal behavior of the sensor and return the value 25.

The mocking mechanism is a crucial part of the multiverse debugger, since it allows the user to explore different execution paths without having to recreate the exact conditions that led to that path. It is therefore not only a part of the automatic traversal of the multiverse tree, but also a part of the manual exploration of the tree. While a user is stepping forwards through a program, and a non-deterministic input primitive is encountered, they can press the *Mock* button below the right pane instead of the *step* button. When doing so, window (4) shown in Figure 1 is used. This

window allows the user to specify that the virtual machine should return a specific value when a primitive is called with certain arguments. This allows users to easily reproduce bugs that might not occur with the sensor values read from the current environment.

Now that we have established how our multiverse debugger works in practice, and its terminology, we will discuss the challenges in creating a multiverse debugger for microcontrollers.

2.5 Challenge C1: Inconsistent External State during Backwards Exploration

When moving backwards in time, not only the internal, but also the external state has to be restored. A multiverse debugger that allows for backwards exploration, but does not handle the external state, will be able to create impossible situations. For instance, if the light sensor program is paused when the red LED has just been turned on, the debugger can move backwards in time to when the light intensity is measured. Reading a different value as you step forwards again, this time, the blue LED could be turned on, even though the red LED is still on. This is an impossible situation, since the LEDs can never be on at the same time in a normal execution of the program. Stepping back over line 21, therefore, requires a compensating action, which turns off the red LED. Luckily, such actions can be defined for many different output operations [52, 82].

2.6 Challenge C2: Exploring Non-deterministic Input in Multiverse Debuggers

When a program's execution path is determined by input from the environment, the multiverse debugger needs to explore different execution paths by traveling back in time and changing the input to the program. However, it is impractical for developers to cover all possible inputs, or to configure the environment exactly right for a specific execution path every time. To solve this problem, we propose a new approach to multiverse debugging that uses time travel debugging in combination with virtual machine instrumentation to mock input values. It is important that no impossible values are used for mocking, since they can lead to inaccessible program states.

2.7 Challenge C3: Keeping Track of the Program State Efficiently

In order for multiverse debugging to work on a microcontroller it has to keep track of the program state and its output effects on the external environment. However, tracking this information on microcontrollers is not feasible due to the limited memory capacity of these devices. Therefore, the MIO debugger is a remote debugger, which enables minimal interference with the microcontroller, and allows information to be stored on a more powerful computer. Even so, it is not feasible to take snapshots for every executed instruction as it would slow down the program significantly, and the size of the snapshot history would quickly become unmanageable.

In time travel debugging, this problem is usually solved by taking snapshots at regular intervals, called checkpoints. When moving backwards in time, the debugger can then jump to the nearest checkpoint, and replay the program's execution from that point. To make a checkpointing system work correctly for multiverse debugging, we had to adopt a slightly different approach. Instead of only taking snapshots at regular intervals, we also take snapshots after each input or output action. By carefully choosing when snapshots are taken we can ensure the correctness of the replay mechanism. Using this approach, we can significantly reduce the run-time overhead, making multiverse debugging practically usable on microcontrollers.

3 A Multiverse Debugger for WebAssembly

In this section, we discuss the operation of our multiverse debugger through a small-step semantic defined over a stack-based language. We use WebAssembly as an example language, since it has a full and rigorous language semantics [32, 78, 79]. However, our small-step rules include very

(1) **Base Language Semantics:** First, define the small-step operational semantics for the base language (Section 3.3). This provides the base model of program execution.
(2) **Well-Behaved I/O:** Extend the base language semantics to ensure all I/O primitives (Section 3.4.1) conform to the requirements for reversible I/O (Section 3.2). This step is crucial for preventing the exploration of inaccessible states.
(3) **Debugger Semantics:** Define the *remote* multiverse debugger in terms of the extended base language semantics. This involves the following steps:
  (a) Define the debugger's state, which encapsulates the program's state in the extended base language (Section 3.5).
  (b) Define forward execution steps for the debugger, incorporating state snapshotting to enable exploration (Section 3.6).
  (c) Define backward execution steps that allow reverting to previously snapshotted states with compensation (Section 3.7).
  (d) Define I/O mocking operations that allow the debugger to control and observe interactions with the environment (Section 3.8).
(4) **Correctness Proof:** Finally, prove the correctness of the debugger (Section 3.10). Specifically, show that the debugger only traverses program states that are reachable during a regular, non-debugging execution.

Fig. 2. Overview of steps to formalize a MIO multiverse debugger for a given base language.

few details specific to WebAssembly. Therefore, we believe that the principles of our multiverse debugger can be applied to any stack-based language, with minimal effort.

3.1 Overview of the Formal Semantics

This section provides a high-level overview of our formalization, abstracting away some of the (important) details. We present this overview in the hope that it makes it easier to apply our techniques to other programming languages. We follow the approach of Torres Lopez et al. [92] in presenting a debugger formalisation recipe. The formalization of our multiverse debugger for an arbitrary language, consists of the steps shown in Fig. 2. For each step, we also indicate the relevant section demonstrating its specific application to WebAssembly.

Base language semantics. First, the small-step operational semantics (SOS) for the base language are defined. This establishes a formal transition system that precisely describes how a program's configuration evolves one computational step at a time. The semantics are defined as a transition relation, relating the program state before taking a step c to the program state after taking a step c'. We will use the following notation, $c \hookrightarrow c'$, where c and c' are program configurations (e.g., the state of the entire program including the stack, memory, etc..). This formal model serves as the base layer upon which all subsequent semantics are built.

Well-behaved I/O. Unconstraint I/O operations present a challenge for reversibility, to address this the base semantics are extended by introducing **compensating actions**. The semantics of each I/O primitive are redefined to return not just a value v, but a pair (v, r), where r is the specific "undo" instruction for that action. For instance, consider a primitive `set_pin_high(13)` that produces a return value v (i.e. a code indicating that the action completed correctly). In our semantics this action is adjusted to also generates its unique compensating action r. This compensating action r is a function that encapsulates the logic for the reverse operation, for example `set_pin_low(13)`.

Debugger semantics. The semantics of the *remote* debugger can be modeled as another transitioning relation $d_c \hookrightarrow_d d'_c$ [1] working over a debugging configuration d_c. This debugging configuration d_c consists of the debugger state (i.e. paused or running), a message box, the underlying program state c and a list of compensating actions. Interestingly the debugger's transition relation, $d_c \hookrightarrow_d d'_c$, is defined in terms of the underlying program's transition relation, $c \hookrightarrow c'$, which it invokes to drive the execution. For example, consider a `step` command issued to the debugger under the form of an incoming message. The debugger, currently in a `paused` state with program state c, will use the underlying base semantics to compute its next state, i.e. it takes one step in the base language semantics $c \hookrightarrow c'$. Once this underlying transition is determined, the debugger completes its own transition, $d_c \hookrightarrow_d d'_c$, by updating its configuration to contain the new program state c'.

During a standard program execution of an I/O action, the forward semantics ($c \hookrightarrow c'$) use the return value v and simply discard the compensating action r [2]. In contrast, in the debugger's semantics these compensating actions are captured. This allows for valid backward transitions, to reverse the `set_pin_high(13)` operation from the example, the debugger executes its saved compensating action, restoring the system to its previous state by setting the pin low.

While this overview already gives a high-level summary of the core semantics, it omits some important details. In the following sections we present the precise inference rules for forward execution with snapshotting (Section 3.6) and backward execution using compensation (Section 3.7) which make our approach scalable for use on microcontrollers. Finally, we will define the semantics for I/O mocking (Section 3.8). We start with detailing the requirements for I/O operations.

3.2 Requirements for I/O Operations

While our multiverse debugger can deal with a large set of I/O operations, there are some limitations that make it impossible to support certain I/O operations. Intuitively, our multiverse debugger supports non-deterministic input primitives as long as the range of possible input values is known. Output primitives are supported as long as they are atomic, and are deterministically reversible, i.e. after reversing an output operation the environment will be in the same state as before applying the operation. In Section 5.4, we discuss the limitations and opportunities for future improvements that these requirements impose on our debugger.

Input primitives. Input primitives are allowed to be non-deterministic as long as the *range* of the input primitives is known, i.e. a temperature sensor might have a range between -20 degrees till 160 degrees. Knowing the range of our input primitives is important so that the debugger can be instrumented to only sample values that can actually be observed during normal execution.

Output primitives. First, we require output primitives to be synchronous and atomic, i.e. all side effects from the operation have to be fully completed during the call of the operation in the virtual machine. Second, for a given execution of the I/O operation, there must be a *deterministic compensating action*. This compensating action reverses the effects of the forward execution, bringing the environment back to a state before executing the actions.

Predictable dependencies. We assume *predictable* dependencies of the I/O operations are known, for example consider a setup where an LED is directly pointed towards a light sensor. During regular execution, turning the LED on will directly influence the possible values which can be read, i.e. there is a dependency between the output pin and the possible sensor values which can be read. Our MIO debugger, has initial support for expressing such simple dependencies, in the semantics however, we take abstraction and assume that sensor values are independent.

[1] Note the d subscript on the arrow

[2] This would of course be optimized in an actual implementation.

(WebAssembly Program state)	K	::=	$\{s, v^*, e^*\}$
(Global store)	s	::=	$\{\text{inst } inst^*, \text{tab } tabinst^*, \text{mem } meminst^*, \text{prim } P\}$
(Primitive table)	P	::=	p^*

(Primitive)	p	:=	$f : v^* \rightarrow \{\text{ret } v, \text{cps } r\}$
(Compensating action)	r	:=	$f : \epsilon \rightarrow \epsilon$

Fig. 3. The configuration for the reversible primitives embedded in the WebAssembly semantics from the original paper by Haas et al. [32], the differences are highlighted in gray. *Top:* The WebAssembly semantics extended with a primitive table. *Bottom:* The signatures of primitive and their compensating actions.

3.3 WebAssembly Language Semantics

WebAssembly is a low-level, portable binary code format designed for safe and efficient execution on various platforms. Its formal semantics are grounded in a stack-based virtual machine, where instructions and values operate on a single stack with strict typing to guarantee fast static validation. We base our formalisation on the semantics from the original WebAssembly paper by Haas et al. [32], where the core semantics include structured control flow (blocks, loops, and conditionals) and memory management via linear memory. WebAssembly intentionally excludes external interface definitions, including I/O operations, to minimize its dependence on platform-specific details. This design choice enables us to deliberately sculpt the I/O operations so that they are deterministically reversible, a necessary condition for multiverse debugging.

The execution of a WebAssembly program is defined by a small-step reduction relation, denoted as $\hookrightarrow_i$ where, i refers to the index of the currently executing module. The relation $\hookrightarrow_i$ is defined over a configuration $\{s; v^*; e^*\}$, with global store s, local values v^*, and the current stack of instructions e^*. The reduction rules take the form $\{s; v^*; e^*\} \hookrightarrow_i \{s'; v'^*; e'^*\}$. Important for our semantics, the global store s contains instances of modules, tables, and memories. The global store allows access to any function within a module instance, denoted as $s_{func}(i, j)$, where i represents the module index and j corresponds to the function index.

3.4 Extending WebAssembly with Primitive I/O Operations

Since multiverse debuggers explore all possible execution paths, they have to be able to reproduce the program's execution, even when non-determinism is involved. It is therefore important to consider where non-determinism is introduced in the program, and how it can be handled. Since the WebAssembly semantics on their own are fully deterministic, we can choose precisely where non-determinism is introduced in our system. In the context of microcontrollers, non-deterministic input is unavoidable. Our system therefore limits non-determinism exclusively to the input[3]. This means that each branch in the execution tree can be traced to a different input value. The output primitives in MIO, on the other hand, are deterministic in terms of the program state. Section 3.8 discuss how the debugger can reproduce non-deterministic input reliably through input mocking.

3.4.1 Primitive I/O Operations. We extend WebAssembly with a set of primitives P that fulfil the prerequisites outlined above. Figure 3 shows the definition of the primitives in WebAssembly. Similar to functions in WebAssembly, each primitive can be identified by a unique index j. Looking

[3]In this work we do not consider parallelism as a source of non-determinism, since this has been examined thoroughly by the original paper on multiverse debugging by Torres Lopez et al. [92].

$$\frac{P(j) = p \qquad p \in P^{In} \qquad v \in \lfloor p(v_0^*)_{ret} \rfloor}{\{s; v^*; v_0^*(call\ j)\} \hookrightarrow_i \{s; v^*; v\}}\ (\textsc{input-prim}) \qquad \frac{P(j) = p \qquad p \in P^{Out} \qquad \lfloor p(v_0^*)_{ret} \rfloor = v}{\{s; v^*; v_0^*(call\ j)\} \hookrightarrow_i \{s; v^*; v\}}\ (\textsc{output-prim})$$

Fig. 4. Extension of the WebAssembly language with non-deterministic input primitives.

(Debugger state)	dbg	::=	$\langle es, msg, mocks, K_n \mid S^* \rangle$
(Execution state)	es	::=	PLAY | PAUSE
(Incoming messages)	msg	::=	$\emptyset \mid step \mid stepback \mid pause \mid play \mid$ $mock \mid unmock$
(Program state)	K	::=	$\{s, v^*, e^*\}$
(Overrides)	$mocks$	::=	$\emptyset$
			$mocks, (j, v^*) \mapsto v$
(A snapshot)	S_n	::=	$\{K_m, p_{cps}\}$
(Snapshots list)	S^*	::=	$S_0 \cdot \ldots \cdot S_{n-1} \cdot S_n$
(Starting state)	dbg_{start}	::=	$\langle \text{PAUSE}, \emptyset, \emptyset, K_0 \mid \{K_0, E\} \rangle$
(Empty action)	r_{nop}	::=	$\lambda().\text{nop}$

Fig. 5. The multiverse debugger state for WebAssembly with input and output primitives.

up primitives is done through the global $P(j)$, and calling a primitive returns both the return value v and the compensating action r. The function r compensates, or reverses, the effects of the primitive, but takes no arguments and returns nothing as indicated by its type $\epsilon \rightarrow \epsilon$. There is no need for any arguments since the compensating action is generated uniquely for each execution of the primitive.

3.4.2 Forwards Execution of Primitives. Given the definition of the primitives in P, we can define the forwards execution of the primitives in WebAssembly, as shown in Fig. 4. Non-determinism is introduced exclusively through the INPUT-PRIM rule, which is used to evaluate input primitives. The evaluation of the primitive p non-deterministically returns a value v from the codomain of the primitive function, $\lfloor p(v_0^*)_{ret} \rfloor$. Here, the rule simply discards the compensating action, and places the return value of the primitive on the stack. The OUTPUT-PRIM rule works analogously, except the evaluation produces its return value deterministically. Note that compensating actions $p(v_0^*)_{cps}$ are not used for the regular forward execution, but are crucial when moving backwards in time. In the next sections, we show how the compensating actions are used during multiverse debugging.

3.5 Configuration of the Multiverse Debugger

Using the recipe for defining debugger semantics from Torres Lopez et al. [92], we can define our multiverse debugger on top of the extended WebAssembly semantics presented in Section 3.4. Figure 5 shows the configuration of the multiverse debugger for WebAssembly with input and output primitives. The program state in the underlying language semantics is labeled with an iteration index n corresponding to the number of steps in the underlying semantic since the start of the execution, or the depth in the multiverse tree. The debugger state dbg contains the execution state of the program, incoming debug message msg, mocked input $mocks$, the program state K_n, and the snapshot list S^*. The snapshots S^* are a cons list of snapshots S_n, containing the program state K_n and the compensating action p_{cps}. The rules presented in the following sections show how the snapshot list is extended—and how the snapshots are used to travel back in time.

$$\frac{\text{non-prim } K_n \qquad K_n \hookrightarrow_i K_{n+1}}{\langle \textsc{play}, \emptyset, mocks, K_n \mid S^* \rangle \hookrightarrow_{d,i} \langle \textsc{play}, \emptyset, mocks, K_{n+1} \mid S^* \rangle} \quad (\textsc{run})$$

$$\frac{\text{non-prim } K_n \qquad K_n \hookrightarrow_i K_{n+1}}{\langle \textsc{pause}, step, mocks, K_n \mid S^* \rangle \hookrightarrow_{d,i} \langle \textsc{pause}, \emptyset, mocks, K_{n+1} \mid S^* \rangle} \quad (\textsc{step-forwards})$$

$$\frac{}{\langle \textsc{play}, pause, mocks, K_n \mid S^* \rangle \hookrightarrow_{d,i} \langle \textsc{pause}, \emptyset, mocks, K_n \mid S^* \rangle} \quad (\textsc{pause})$$

$$\frac{}{\langle \textsc{pause}, play, mocks, K_n \mid S^* \rangle \hookrightarrow_{d,i} \langle \textsc{play}, \emptyset, mocks, K_n \mid S^* \rangle} \quad (\textsc{play})$$

Fig. 6. The small-step rules describing forwards exploration in the multiverse debugger for WebAssembly instructions without primitives.

Mocked inputs are stored as a key value pairs, where the index identifying the input primitive j, and the list of argument values v^* are mapped to the overriding return value v. The key value map is represented here as a partial function, which compares lists of values v^* element-wise. For any key that is not defined in the map, we write $mocks(j, v^*) = \varepsilon$.

The starting state of the debugger dbg_{start} is defined as the paused state with no incoming or outgoing messages, an empty mocks environment, the initial program state K_0, and a snapshot list containing only the initial snapshot $S_0 = \{K_0, r_{nop}\}$. Here r_{nop} is the empty action, which takes no arguments and returns nothing. This function indicates that no compensating action is needed.

3.6 Forwards Exploration in Multiverse Debuggers

Figure 6 shows the basic small-step rules for stepping forwards in the multiverse debugger, without input and output primitives. These rules allow the debugger to explore traditional WebAssembly programs, without any non-deterministic input or output. For clarity, we use several shorthand notations in the rules. We use the notation $(K_n \hookrightarrow_i K_{n+1})$ to say that the program state K takes a step to the program state K' in the underlying language semantics, where $K' = K_{n+1}$. The notation (non-prim K) is used to indicate that the program state K is not a primitive call, or more fully, it is not the case that $K = \{s; v^*; v_0^*(call\ j)\} \wedge P(j) = p$. We describe the rules in detail below.

- **RUN** The rule for running the program forwards in the underlying language semantics. The debugger takes a step in the underlying language semantics ($K_n \hookrightarrow_i K_{n+1}$) as long as the execution state is PLAY, there are no incoming or outgoing messages, and the program state is not a primitive call. While running in this way, the snapshot list S^* remains unchanged.
- **STEP-FORWARDS** When the debugger receives the *step* message, it takes one more step ($K_n \hookrightarrow_i K_{n+1}$), and transitions to the PAUSE state if it was not already paused.
- **PAUSE** When the debugger receives a *pause* message in the PLAY state, it transitions to the PAUSE state. Note that afterwards, all the run rules are no longer applicable.
- **PLAY** The rule for continuing the execution. When the debugger receives the *play* message in the PAUSE state, the execution state transitions to the PLAY state.

$$\frac{K_n = \{s; v^*; v_0^*(call\ j)\} \qquad P(j) = p \qquad p \in P^{In} \qquad mocks(j, v_0^*) = \varepsilon \qquad K_n \hookrightarrow_i K_{n+1}}{\langle \textsc{play}, \varnothing, mocks, K_n \mid S^* \rangle \hookrightarrow_{d,i} \langle \textsc{play}, \varnothing, mocks, K_{n+1} \mid S^* \cdot \{K_{n+1}, r_{nop}\} \rangle} \quad (\textsc{run-prim-in})$$

$$\frac{\begin{array}{c} K_n = \{s; v^*; v_0^*(call\ j)\} \\ P(j) = p \qquad p \in P^{Out} \qquad p(v_0^*) = \{\text{ret } v, \text{cps } r\} \qquad K_{n+1} = \{s; v^*; v\} \end{array}}{\langle \textsc{play}, \varnothing, mocks, K_n \mid S^* \rangle \hookrightarrow_{d,i} \langle \textsc{play}, \varnothing, mocks, K_{n+1} \mid S^* \cdot \{K_{n+1}, r\} \rangle} \quad (\textsc{run-prim-out})$$

Fig. 7. The small-step rules describing forwards exploration for input and output primitives in the multiverse debugger for WebAssembly, without input mocking.

The rules in Fig. 7 are the minimal set of rules for stepping forwards when the next instruction in the program state K_n is a primitive call. The rules also describe the snapshotting behavior of the multiverse debugger, which is needed to travel back in time. We describe the run rules in detail below. The step rules are identical to the run rules, but they transition to the paused state after taking a step, and are triggered by the *step* message. These rules can be found in the online appendix and the extended version [55].

RUN-PRIM-IN The rule for calling an input primitive. When the program state is a primitive call, the call will not be mocked, there are no incoming messages, and the execution state is PLAY, the debugger takes a step forwards in the underlying language semantics ($K_n \hookrightarrow_i K_{n+1}$), and adds a new snapshot to the snapshot list $\{K_{n+1}, r_{nop}\}$. The compensating action r_{nop} is used to indicate that no compensating action is needed for input primitives.

RUN-PRIM-OUT The rule for calling an output primitive. When the program state is an output primitive call, there are no incoming messages, and the execution state is PLAY, the debugger will perform the primitive call similar to the OUTPUT-PRIM rule in the underlying language semantics. It adds the return value of the primitive to the stack, moving the state to K_{n+1}, the same state reached by the underlying language semantics. However, it will not discard the compensating action $p(v_0^*)_{cps} = r$, but add a new snapshot to the snapshot list $\{K_{n+1}, r\}$.

The rules described here form the backbone of the multiverse debugger, and already allow for a multiverse debugger that can explore the execution of a program forwards in time. It is important to note that the debugger always takes snapshots when a primitive call is made. Only when this primitive is an output primitive, can the state of the environment change. On the other hand, only input primitives can introduce new branches to the multiverse tree. In the next section, we discuss how the multiverse debugger can explore the execution of a program backwards in time.

3.7 Backwards Exploration with Checkpointing

The multiverse debugger is also a time travel debugger, which means it can move backwards in time. It does this by restoring the program state from a previous snapshot, and then replaying the program's execution from that point. Figure 8 shows schematically the way the debugger steps back in time using the snapshots. In the situation depicted by the figure, the debugger has just stepped over an output primitive and added a snapshot to the snapshot list. Since the debugger will now step back over the execution of this output primitive, its effects must be reversed with the compensating action r'' in the last snapshot. This is shown with the dashed arrow. As part of this jump back in time, the snapshot containing the compensating action r'' is removed from the snapshot list and the program state K_n from the next snapshot is restored.

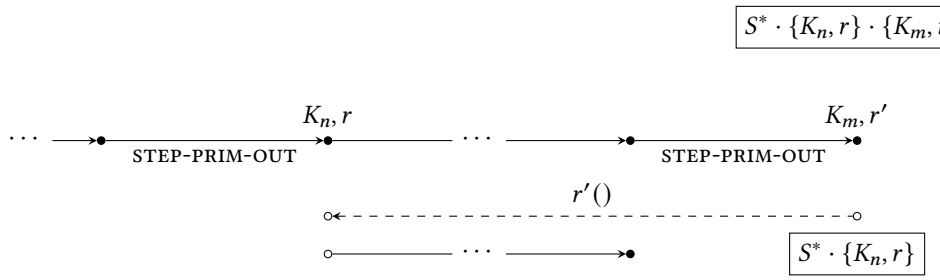


Fig. 8. Schematic of the STEP-BACK-COMPENSATE rule applied in state K_m. The dotted arrow shows how the debugger jumps to the previous state K_n, and compensates the output primitive with $r'()$, while the full arrows show the normal execution. Top right: the snapshots before. Bottom right: the snapshots after.

(STEP-BACK)

$$\frac{K_n \hookrightarrow_i^{m-n-1} K_{m-1} \qquad m > n}{\langle \text{PAUSE}, \mathit{stepback}, \mathit{mocks}, K_m \mid S^* \cdot \{K_n, r\}\} \rangle \hookrightarrow_{d,i} \langle \text{PAUSE}, \varnothing, \mathit{mocks}, K_{m-1} \mid S^* \cdot \{K_n, r\} \rangle}$$

(STEP-BACK-COMPENSATE)

$$\frac{\text{first } r'() \text{ then } K_n \hookrightarrow_i^{m-n-1} K_{m-1}}{\langle \text{PAUSE}, \mathit{stepback}, \mathit{mocks}, K_m \mid S^* \cdot \{K_n, r\} \cdot \{K_m, r'\} \rangle \hookrightarrow_{d,i} \langle \text{PAUSE}, \varnothing, \mathit{mocks}, K_{m-1} \mid S^* \cdot \{K_n, r\} \rangle}$$

Fig. 9. The small-step reduction rule for stepping backwards in the multiverse debugger.

Since snapshots are only added when the program performs a primitive call, the second to last snapshot in the list was taken after the previous primitive call resulting in state K_n. This means, that after restoring the internal virtual machine state, the program is now at the point right after the previous primitive call. Starting from this point, the debugger can replay the program's execution forwards to K_{m-1}, which will not include any primitive calls. This means the steps will be deterministic, and will not change the external environment. This corresponds with the full arrow at the bottom of the figure. Next to it is shown the snapshot list after the step back, which now only contains the snapshot of the state K_n.

In the outlined scenario the last transition in the chain, from K_{m-1} to K_m, performs an output action. However, if this transition is a standard WebAssembly instruction instead, no compensating action will be performed. Instead, the debugger will immediately restore the virtual machine state to K_n and replay the program's execution forwards to K_{m-1}. In this case, none of the snapshots will be removed. This enables the debugger to continue stepping back in time.

Each of the two outlined scenarios correspond with a rule in the multiverse debugger semantics, shown in Fig. 9. The first scenario where the effects of an output primitive is reversed, is described by the STEP-BACK-COMPENSATE rule. The second scenario where the last transition is a standard WebAssembly instruction, is described by the STEP-BACK rule. We describe the rules in detail below.

STEP-BACK The rule for stepping back in time. When the debugger receives a *step back* message, the debugger restores the external state from the last snapshot in the snapshot list, which is not the current state. The debugger then replays the program's execution from that point to exactly one step ($K_n \hookrightarrow_i^{m-n-1} K_{m-1}$) before the starting state. Since the restored snapshot remains in the past, it is kept in the snapshot list, to allow for further backwards exploration.

$$\text{(REGISTER-MOCK)}\quad \frac{msg = mock(j, v^*, v) \qquad P(j) = p \qquad p \in P^{In} \qquad v \in \lfloor p \rfloor \qquad mocks' = mocks, (j, v^*) \mapsto v}{\langle rs, msg, mocks, K_n \mid S^* \rangle \hookrightarrow_{d,i} \langle rs, \emptyset, mocks', K_n \mid S^* \rangle}$$

$$\text{(UNREGISTER-MOCK)}\quad \frac{msg = unmock(j, v^*) \qquad mocks' = mocks \setminus (j, v^*) \mapsto v}{\langle rs, msg, mocks, K_n \mid S^* \rangle \hookrightarrow_{d,i} \langle rs, \emptyset, mocks', K_n \mid S^* \rangle}$$

$$\text{(STEP-MOCK)}\quad \frac{K_n = \{s; v^*; v_0^*(call\ j)\} \qquad P(j) = p \qquad p \in P^{In} \qquad mocks(j, v_0^*) = v \qquad K'_{n+1} = \{s'; v'^*; v\}}{\langle \text{PAUSE}, step, mocks, K_n \mid S^* \rangle \hookrightarrow_{d,i} \langle \text{PAUSE}, \emptyset, mocks, K'_{n+1} \mid S^* \cdot \{K'_{n+1}, r_{nop}\} \rangle}$$

Fig. 10. The small-step rule for mocking input in the MIO debugger, only including the step rule. The analogous rule for when the debugger is not paused (RUN-MOCK) can be found in the online appendix and the extended version [55].

STEP-BACK-COMPENSATE The rule for stepping back in time when the last transition was a primitive call. This is always the case when the current state is K_m is part of the last snapshot. When the debugger receives a *step back* message, the debugger performs the compensating action r' from the last snapshot in the snapshot list, which reversed the effects of the last primitive call. Then, the debugger restores the external state K_n from the second to last snapshot in the snapshot list. The debugger then replays the program's execution from that point to exactly one step before the starting state. The last snapshot is removed from the snapshot list, since it now lies in the future.

In the case, where no primitive call has yet been made, the snapshot list contains exactly $\{K_0, r_{nop}\}$, as defined by dbg_{start}, which the STEP-BACK rule can jump to. If the current state is K_0, stepping back is not possible. Specifically, the STEP-BACK rule is not be applicable, since m and n are both zero, and the STEP-BACK-COMPENSATE rule requires the snapshot list to contain at least two snapshots.

3.8 Instrumenting Non-deterministic Input in Multiverse Debuggers

In order to replay execution paths in the multiverse tree accurately, the multiverse debugger needs to be able to override the input to the program. This is done through the *mocking* of input, described by the rules in Fig. 10. Mocking of input happens through the key value map *mocks* shown in Fig. 5. New values can be added to the map using the REGISTER-MOCK rule, and existing values can be removed using the UNREGISTER-MOCK rule. Whenever the debugger encounters an input primitive call, it will always check the *mocks* map for an overriding value. If a value is found, the debugger will replace the call to the primitive with the mock value v. This is done by the STEP-MOCK rule.

REGISTER-MOCK The rule for registering a new mock value in the multiverse debugger. When the debugger receives a message $mock(j, v^*, v)$, it updates the entry for (j, v^*) in the *mocks* environment to v. If an entry already exists, the rule will override the existing value.

UNREGISTER-MOCK The rule for unregistering a mock value in the multiverse debugger. When the debugger receives a message $unmock(j, v^*)$, it removes the mock value from the *mocks* map. If no value is found in the environment, the rule will have no effect.

STEP-MOCK The STEP-MOCK rule for stepping forwards in the multiverse debugger when an input primitive call is encountered. If the input primitive call is found in the *mocks* map, the debugger will replace the call with the mock value v. The program state is then updated to the new program state K_{n+1}, and a new snapshot is added to the snapshot list. The snapshot includes the new program state and the empty compensating action r_{nop}, since no compensating action is needed for input primitives.

3.9 Arbitrary Exploration of the Multiverse Tree

With the semantics of input mocking in place, we now have the entire multiverse debugger semantics for WebAssembly with input and output primitives. In this section, we discuss how the multiverse debugger can be used to explore different universes. This can be done by Algorithm 1. When the debugger jumps from a state K_m to a state K_n, the debugger will find the smallest common ancestor of K_m and K_n, or the join. The debugger will then step backwards from K_m to the join. We use the notation $\hookrightarrow_r$ to indicate that the debugger is reversing the execution, it is equivalent to a debugging step $\hookrightarrow_{d,i}$ that only uses the STEP-BACK and STEP-BACK-COMPENSATE rules. In the final step of the algorithm, execution is replayed from the join to K_n using the STEP-MOCK rule whenever it encounters a non-deterministic primitive call.

Algorithm 1: The algorithm for traveling to any position in the multiverse tree.

Data: The current program state K_m, the target program state K_n, and the snapshot list S^*.
$K_{\text{join}} \leftarrow \text{find_join}(K_m, K_n)$;
while $dbg_{current}[K] \neq K_{join}$ **do**
  $dbg_{current} \hookrightarrow_r dbg_{next}$;
  $dbg_{current} \leftarrow dbg_{next}$;
end
while $dbg_{current}[K] \neq K_n$ **do**
  $dbg_{current} \hookrightarrow_{d,i} dbg_{next}$;
  $dbg_{current} \leftarrow dbg_{next}$;
end

Figure 11 illustrates the algorithm for jumping to an arbitrary state, when the user clicks on a node on another branch in the multiverse tree. The figure shows a possible multiverse tree for a program where the second and third instruction are input primitives. The program has executed two input primitives in a row, and the debugger has explored some of the possible inputs. Each node in the figure is labeled with the program state and possible compensating action, where r_{nop} indicates that no compensating action is needed. For clarity, the external state are also numbered. The figure shows clearly that the external state only changes after a primitive call. The current state is K_5, and the debugger wants to jump to K'_4. Per the algorithm, the debugger finds the join of the two states, which is K_1. The debugger then replays the execution from K_5 to K_1 in reverse order, using the STEP-BACK and STEP-BACK-COMPENSATE rules. It is important that the debugger steps back one instruction at a time, to ensure that the external state is correctly restored. From the join K_1, the debugger replays the execution to K'_4 in the forward order, using the STEP-MOCK rule whenever it encounters a non-deterministic primitive call. This ensures that the jump deterministically follows the exact execution path, thereby ensuring that the external state is correctly restored.

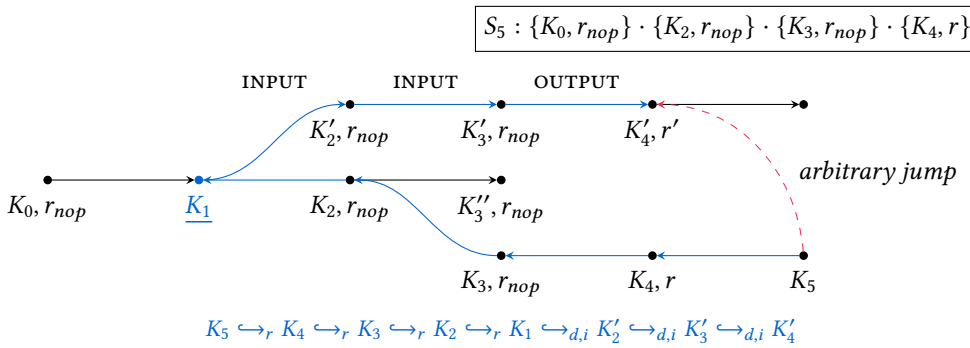


Fig. 11. Schematic of how the multiverse debugger can jump to any arbitrary state in the past, using the STEP-BACK and STEP-MOCK rules. For the arbitrary jump from state K_5 to K'_4, the join K_1 is underlined and shown in blue. Top right: the list of snapshots before the arbitrary jump. Bottom: the execution path from K_5 to K'_4. Steps with the STEP-BACK and STEP-BACK-COMPENSATE rules are shown as $\hookrightarrow_r$.

3.10 Correctness of the Multiverse Debugger Semantics

Given the small-step semantics, we can prove the correctness of the MIO debugger in terms of soundness and completeness. The soundness theorem states that for any debugging session ending in a certain state, there also exists a forwards execution path in the underlying language semantics to that state. A debugging session is seen as any number of debugging steps starting from the initial debugging state dbg_{start}. The completeness of the debugger means that the debugger can always find a path in the multiverse tree that corresponds to a path in the underlying language semantics. Together, these properties ensure that the debugger is correct in terms of its observation of the underlying language, and will never observe any inaccessible states. For brevity, we only provide a sketch of the proofs here, but the full proofs can be found in the online appendix and the extended version [55].

THEOREM 1 (DEBUGGER SOUNDNESS). *Let K_0 be the start WebAssembly configuration, and dbg the debugging configuration containing the WebAssembly configuration K_n. Let the debugger steps $\hookrightarrow^*_{d,i}$ be the result of a series of debugging messages. Then:*

$$\forall dbg : dbg_{start} \hookrightarrow^*_{d,i} dbg \Rightarrow K_0 \hookrightarrow^*_i K_n$$

The proof for debugger soundness proceeds by induction over the number of steps in the debugging session. In the base case, where the debugging session consists of a single step, the proof is trivial since the step starts from the initial state. In the inductive case, the proof proceeds very similarly, the only non-trivial cases are those for stepping backwards and mocking.

THEOREM 2 (DEBUGGER COMPLETENESS). *Let K_0 be the start WebAssembly configuration for which there exists a series of transition $\hookrightarrow^*_i$ to another configuration K_n. Let the debugging configuration with K_n be dbg. Then:*

$$\forall K_n : K_0 \hookrightarrow^*_i K_n \Rightarrow dbg_{start} \hookrightarrow^*_{d,i} dbg$$

The proof for completeness follows almost directly from the fact that for every transition in the underlying language semantics, the debugger can take a corresponding step. For non-deterministic input primitives, we can step to the same state with the REGISTER-MOCK and STEP-MOCK rules.

Together the debugger soundness and completeness theorems ensure that the multiverse debugger is correct in terms of its observation of the underlying language semantics. However, it

gives us no guarantees about the correctness of the compensating actions, and the consistency of external effects during a debugging session. Due to the way effects on the external environment are presented in the MIO debugger semantics, we can define the entire effect of a debugging session of regular execution, both as ordered lists of steps that have external effects. There are only two options, the output primitive rules, and the rule that applies the compensating action.

Definition 3 (External state effects). The function *external* returns the steps affecting external state for any series of rules in the debugging or underlying language semantics.

$$external(p) = \begin{cases} (s \text{ for } s \text{ in } p \text{ where } s = \textsc{step-prim-out} & \text{if } p = dbg \hookrightarrow^*_{d,i} dbg' \\ \vee\ s = \textsc{step-back-compensate}) & \\ (s \text{ for } s \text{ in } p \text{ where } s = \textsc{output-prim}) & \text{if } p = K \hookrightarrow^*_i K' \end{cases}$$

Using this definition, we can prove that the external effects of any debugging session ending in a certain state, are the same as the effects of the regular execution of the program ending in that same state. The definition for the equivalence of external effects ($\equiv$) is given in the online appendix and the extended version [55].

Theorem 4 (Compensation soundness). *Let K_0 be the start WebAssembly configuration, and dbg the debugging configuration containing the WebAssembly configuration K_n. Let the debugger steps $\hookrightarrow^*_{d,i}$ be the result of a series of debugging messages. Then:*

$$\forall dbg : external(dbg_{start} \hookrightarrow^*_{d,i} dbg) \equiv external(K_0 \hookrightarrow^*_i K_n)$$

The proof of this theorem is based on the fact that our multiverse debugger is a rooted acyclic graph, and a debugging session is a walk in this tree starting from the root, which can include the same edge several times. Any such walk in a tree can be constructed by adding any number of random closed walks to the path from the root to the final node. Such closed walks are null operations in terms of their effect on the external state. This leaves only the forward steps of the minimal path to be considered, meaning the external effects of a debugging session are always the same as those of the regular execution of the program.

4 The MIO Debugger

We have implemented the multiverse debugger described above in a prototype debugger, called the MIO debugger. The MIO debugger is built on top of a WebAssembly runtime for microcontrollers, called WARDuino [54]. WARDuino is written in C++, includes primitives for controlling hardware peripherals, and has support for traditional remote debugging. Our prototype implementation builds further on its virtual machine and the remote debugging facilities but needed to be extended significantly in order to support all the basic operations for multiverse debugging: smart snapshotting, mocking of primitives and reversible actions. Additionally, we created a high-level interface which implements the message passing interface described in Section 3 as messages in the remote debugger of WARDuino. On top of this interface we built a Kotlin application for debugging AssemblyScript programs on microcontrollers running WARDuino. This application keeps track of the program states, and shows them as part of the multiverse tree, as shown in Fig. 1 from Section 2. Algorithm 1 for arbitrary jumping, is implemented at the level of the Kotlin application using the message passing interface of the remote debugger. Finally, the MIO prototype also has support for expressing simple dependencies, so that the mocking of the various sensor values can be limited depending on state of the output pins.

```
1  def_prim(rotate, threeToNoneU32) {
2    int32_t speed = arg0.int32;
3    int32_t degrees = arg1.int32;
4    int32_t motor = arg2.int32;
5    pop_args(3);
6    auto encoder = encoders[motor];
7    encoder->set_angle(
8      encoder->get_angle() + degrees
9    );
10   return drive(motor, encoder, speed);
11 }
```

```
10 def_prim_serialize(rotate) {
11   for (int m = 0; m < MOTORS; i++) {
12     external_states.push_back(
13     new MotorState(m, encoders[m]->angle()));
14 }}
15
16 def_prim_reverse(rotate) {
17   for (IOState s : external_states) {
18     if (isMotorState(s)) {
19       int motor = stoi(s.key);
20       auto encoder = encoders[motor];
21       encoder->set_angle(s.degrees);
22       drive(motor, encoder, STD_SPEED);
23 }}}
```

Fig. 12. *Left:* The implementation of the *rotate* primitive. *Right:* The implementation of the compensating action for the *rotate* primitive, in the MIO debugger.

4.1 Output: Reversible Primitives

Primitives in WARDuino are implemented in the virtual machine using C macros. In order to implement reversible primitives, we have extended the existing macros with two new macros; one defines how the external state effected by the primitive can be captured, and the other defines the compensating action given this captured state. When stepping back over a primitive, the compensating action looks at the state captured after the previous primitive call, and restores this external state. This is the same as undoing the effects of the last primitive call.

To illustrate the implementation of reversible primitives, we will use the example of the *rotate* primitive, which rotates a servo motor for a given number of degrees. The forwards implementation is shown on the left side of Listing 12. To move the motor a given number of degrees the primitive first sets the target angle of the motor encoder, this happens on line 7. The motor encoder is used to track the current motor angle, as well as the absolute target angle, which can be set with the *set_angle* method. To rotate the motor a number of degrees relative to its current position, the primitive adds the degrees to the current motor angle (line 8). Once the target angle is set, the primitive drives the motor to that angle using the *drive* method, as shown on line 10.

The implementation of the compensating action for the *rotate* primitive is shown on the right side of Listing 12. First, the *def_prim_serialize* macro captures the external state. For each motor, the current angle of the motor is stored along with its index, as shown on line 13. Second, the *def_prim_reverse* macro compensates the primitive by moving all motors back to the angles captured in the previous snapshot. The angles captured by the *def_prim_serialize* macro are absolute target angles. The compensating action moves the motors back to these angles by first setting the target angle, as shown on line 21. It then uses the same *drive* function to move the motor.

4.2 Input: Mocking of Primitives

The input mocking is implemented analogous to the debugger semantics, by adding a map to the in the virtual machine state. This map is used to store the mocked values for the input primitives, which are received by a new debug message in the remote debugger. In line with the semantics, there is also a new debug message to remove a mocked value from the map. Currently, the map only supports registering primitive calls with their first argument. This is sufficient for the current input primitives to be mocked, without any changes to their implementation.

The virtual machine will check the map of mocked values for every primitive call. The prototype includes two input primitives that can be mocked in this way, the *digitalRead* primitive which reads the value of a digital pin, and the *colorSensor* primitive which reads a value from a uart color sensor. The digitalRead primitive enables the user to mock the value of a digital pin, and thereby the behavior of a wide range of possible peripherals. However, the range of possible input values is not always known statically, as it may be influenced by the output effects of the program. To handle this, the MIO debugger includes initial support for predictable dependencies that can be defined as simple conditions, for example, *"when the value of a digital pin n is x, then input primitive p with arguments m will return the value c"*.

4.3 Performance: Checkpointing

To reduce memory usage, the MIO debugger only stores snapshots at certain checkpoints. The semantics of MIO only takes snapshots after a primitive call, the prototype implementation follows this checkpointing policy precisely. As shown by the debugger semantics and the proof, this is the minimum number of snapshots needed to enable backwards and forwards exploration of the multiverse tree. To further reduce the performance impact on the microcontroller, snapshots are received and tracked by the desktop frontend of the MIO debugger. To have minimal traffic between the backend and frontend, snapshots after primitive calls are sent automatically to the frontend. Alternatively, the frontend can request snapshots at will through the remote debugger interface.

5 Evaluation

To validate that our checkpointing strategy is performant enough for apply multiverse debugging on microcontrollers we performed a number of experiments. All experiments were performed on an STM32L496ZG microcontroller running at 80 MHz. This microcontroller was connected to a laptop running the MIO debugger frontend that communicates with the microcontroller.

5.1 Forward Execution with Checkpointing

The first experiment evaluates the performance impact of checkpointing on the execution speed. We measured the execution time of a fixed number of instructions, when taking no snapshots, taking a snapshot every instruction, and for snapshotting after different intervals (5, 10, 50, or 100 instructions), as shown in Fig. 13. To reduce the impact of variable unknown factors, the program executed by the virtual machine includes no primitive calls. Specifically, this program checks for each integer from 1 to 13,374,242 if they are prime. Because this program has no primitive calls, the VM will only take snapshots at fixed intervals which are determined by the frontend.

The left plot shown in Fig. 13, gives the time it took to execute up to 1250 instructions for each snapshot policy relative to taking no snapshots. Since snapshotting at every instruction is so much slower, we show the same zoomed in results in the right plot, but omit the results for snapshotting at every instruction. For such small numbers of instructions, the execution time without any debugger intervention, remains roughly the same, taking on average 222.7ms. These results are shown in red. In contrast, when taking snapshots after every executed instruction, the execution time increases dramatically. For 1250 instructions it takes on average 19 seconds, which is around 85 times slower. For only 250 instructions the execution time increases seventeen-fold, to 3.9 seconds.

Once checkpointing is used the overhead reduces significantly. When taking snapshots every five instructions, the virtual machine only needs 4 seconds to execute 1250 instructions. Per instruction this results in an execution that is only 17.9 times slower. Taking a snapshot every 10 instructions results in a total execution time of 2.1 seconds. This results in a slowdown of factor 9.5. When taking snapshots every 50 instructions, the slowdown lowers to a factor of 2.7. Going up to a hundred instructions every snapshot, this becomes only a factor 1.9.

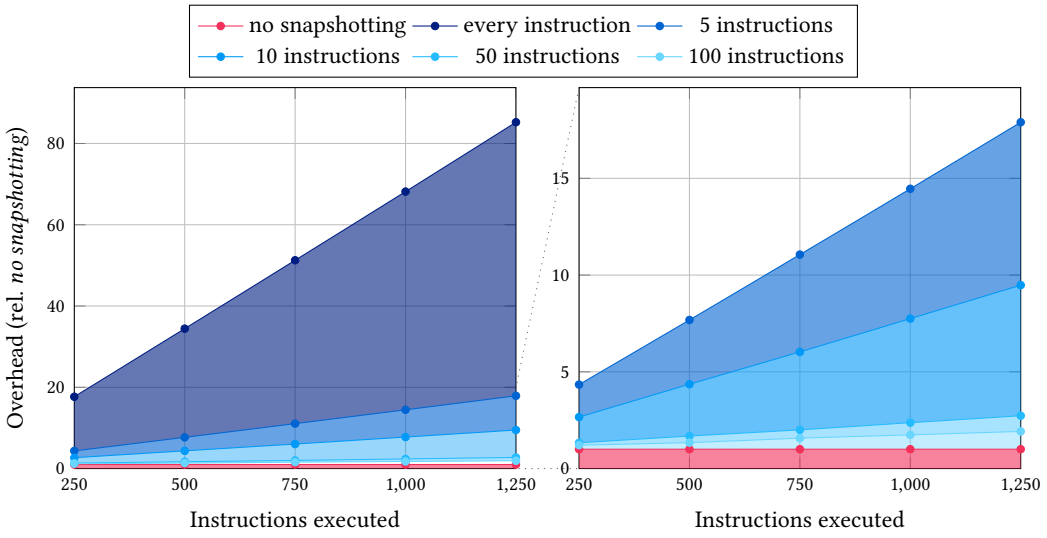


Fig. 13. Comparison of execution time of snapshotting: *never*, at *every instruction*, and checkpointing intervals; *5, 10, 50, and 100 instructions*. The performance overhead is shown as execution time relative to the execution time when taking no snapshots. Left: Comparison of all policies. Right: The performance of the policies excluding snapshotting every instruction (zoom-in). The averages are taken over 10 runs of the same program.

This initial benchmark of the checkpointing strategy shows that the performance overhead can be greatly reduced by reducing the number of snapshots taken. Yet, execution times are still significantly slower than without any snapshotting. This is due to the fact that the current prototype has not yet been optimized for performance. The prototype only uses simple run-length encoding of WebAssembly memory to reduce the size of the snapshots. In future, the snapshot sizes could be reduced greatly by only communicating the changes compared to the previous snapshot. However, in practice the performance is already sufficient to provide a responsive debugger interface as we illustrate in the online demo videos, which can be found here[4]. The example we highlight later in Section 5.3, requires on average snapshot every 37 instructions. This reduces overhead sufficiently to have a responsive debugging experience for users. Additionally, the I/O operations by comparison typically take much longer to execute, a single action easily taking several seconds.

5.2 Backwards Execution

The graphs in Fig. 13 only show part of the picture, where the less snapshots are taken, the better the performance. Unfortunately, there is no such thing as a free lunch, and while only taking one snapshot at the start of the program and never again, would result in the lowest possible overhead for forwards execution, this is not the case for backwards execution. In that case stepping back would always have to re-execute the entire program. Clearly, the further apart the snapshots, the longer it will take to step back. To illustrate this trade-off, we examined the impact of the number of re-executed instructions on stepping back speed.

Figure 14 shows the average time it takes to step back as the number of instructions requiring re-execution increases in increments of one thousand. The averages are calculated over 10 runs of the same program used in the previous section. When executing only a handful of instructions, the

[4]Full link: https://youtube.com/playlist?list=PLaz61XuoBNYVcQqHMAAXQNf8fz5IAMahe&si=HNrKY9YzqDFadATN

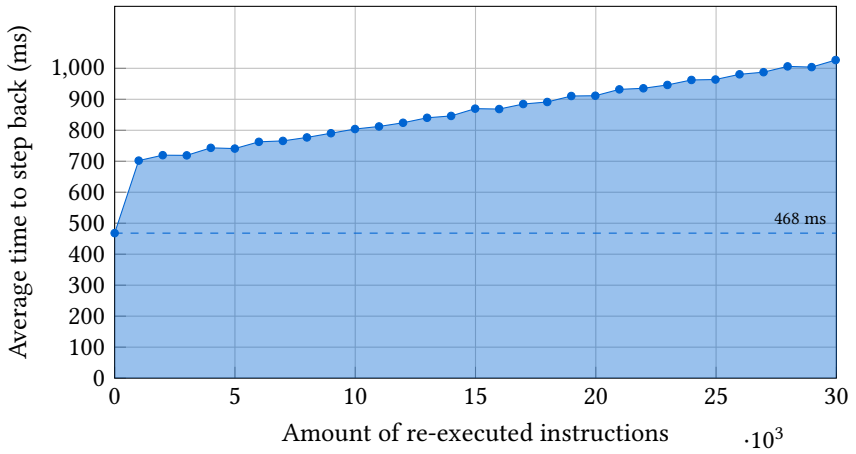


Fig. 14. Plot showing the average time to step back as the number of instructions requiring re-execution increases in increments of one thousand. Averages are calculated over 10 runs of the same program.

time to step back is dominated by the communication latency between the microcontroller and the debugger frontend. On average, this results in a minimal time of 468ms to step back. Between one thousand and 30 thousand re-executed instructions, the time to step back increases linearly by roughly 11ms per a thousand instructions.

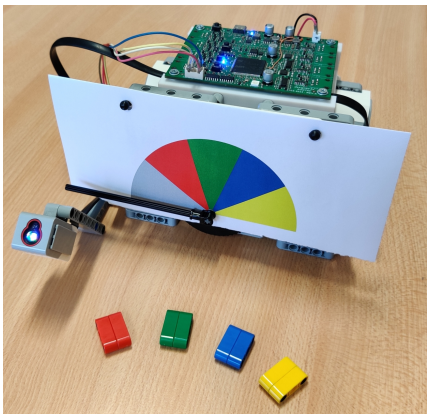
Our analysis of the checkpoint strategy's impact on stepping back shows that the overhead is minimal. The prototype is able to re-execute 30 thousand non-I/O instructions in around one second. Compared to the overhead of checkpointing on forwards execution (see Fig. 13), we can safely conclude that in practice the overhead on backwards execution is negligible. This is further evidenced in our demo videos, where developers mostly have to wait for physical I/O actions to complete, and stepping back is otherwise instantaneous.

5.3 Use case: Lego Mindstorms Color Dial

To illustrate the practical potential of MIO and its new debugger approach, we present a simple reversible robot application using Lego Mindstorms components. However, not just microcontroller applications may benefit from our novel approach, there are many application domains where output is entirely in the form of digital graphics, which are more easily reversible—such as video games, simulations, etc. Nevertheless, to highlight the potential of the approach we demonstrate the MIO debugger using small physical robots and other microcontroller applications, as this is a more challenging environment for multiverse debugging. Using the digital input and motor primitives described in Section 4, we developed a color dial, as a simplified application. We developed this example alongside a few others to further demonstrate the usability of the MIO debugger[5], and have created demo videos for a few of the examples, which can be found online[6].

The color dial application works as follows, the robot has a color sensor that can detect the color of objects. Depending on the color seen by the sensor, a single motor will move the needle on the dial to the location indicating the color seen by the sensor. We built the dial using LEGO Mindstorms components [23] as shown on the left of Figure 15. The right-hand side of Fig. 15 shows the infinite loop that controls the robot, written in AssemblyScript. In this loop, the robot

[5]Code for all examples can be found *[link to repository removed for double-blind review]*
[6]Full link: https://youtube.com/playlist?list=PLaz61XuoBNYVcQqHMAAXQNf8fz5IAMahe&si=HNrKY9YzqDFadATN



```
1  enum Color { none = 0, red = 1, green = 2,
2               blue = 3, yellow = 4 }
3
4  const sensor = colorSensor(Pin.IO2);
5  let current: Color = Color.none;
6  while (true) {
7    let next: Color = sensor.read();
8    if (next != current) {
9      // turn the needle if the color changed
10     rotate(Pin.IO1,
11       (next - current) * angle, speed);
12   }
13   current = next;
14 }
```

Fig. 15. Left: Lego color dial that recognizes the color of objects. Right: The main loop controlling the color dial. The dial has a single motor connected to pin IO1, and the color sensor is connected to pin IO2.

will continually read sensor values from the color sensor. While doing so it will move the needle of the dial to the correct position indicating the current color seen by the sensor. The needle is only moved if the color sensor sees a value different from what the dial is currently indicating. The relative amount that the needle needs to move is calculated by taking the difference between the current color the needle is pointing at and the new color.

The program for the color dial uses the reversible primitive *rotate*, used as an example in Section 4, to rotate the needle of the dial. By using only reversible output primitives, the program written for this robot automatically becomes reversible. This means that while debugging the application, the color the needle is pointing towards will always correspond to variable *current* in the program. Concretely, if the debugger steps back through the program from the end of a loop iteration to line 8, it will move the needle back to the previous color without having to read a new sensor value. This makes it easy to test certain state transitions where the needle is pointing at one particular color and now has to move to a different color.

Aside from using time travel debugging which keeps external state in mind, users of our debugger are also able to leverage multiverse debugging capabilities to deal with the non-deterministic nature of this color sensor. This allows them to easily simulate various sensor values, and explore the different paths the robot can take without needing to use any real, correctly-colored objects. This example touches on a few common aspects of robotics applications, such as processing non-deterministic input, controlling motors and making decisions based on sensor values. Using the I/O primitives supported by MIO, various other applications could be build; such as a binary LED counter, a smart curtain, an analogue clock, a maze solving robot, and so on.

5.4 Discussion and Future Work

Our debugger's formal guarantees are built upon a constrained subset of general I/O operations i.e. deterministically reversible operations. These operations set the boundaries of where we can guarantee that our debugger is operating correctly. The primary limitations in comparison with general I/O operations concern inputs with unbounded or unknown ranges, such as raw network sockets or file systems, as these prevent the exhaustive instrumentation required by our debugger. Furthermore, our system excludes outputs that are asynchronous (e.g., an asynchronous web-api)

or physically irreversible (e.g., firing a missile), as these violate the core requirements for atomic execution and deterministically reversible actions.

Future research could substantially extend or relax these boundaries. For instance, the challenge of unbounded inputs could be addressed through user-defined sampling strategies that allow developers to specify relevant input subsets for exploration. These sampling strategies could potentially provide a practical and usable system, even though they don't cover every possibility. The constraint of physical reversibility could be relaxed by operating in a more controllable environment where reversibility is pushed further. The requirement for having external output to be synchronous was mostly introduced to keep the formalization of the system from growing too complex. Independent asynchronous output actions could thus potentially be supported but would require a substantial formalization effort. Finally, a formal language for declaring I/O interdependencies would allow the debugger to reason about more complex, realistic hardware setups, moving beyond our current simple interdependency assumptions.

Despite these potential extensions, some operations remain fundamentally incompatible with the reversible multiverse debugging paradigm. The ability to reverse an execution is essential, and this is impossible for operations causing an irreversible state change in an uncontrolled *external* environment. Actions such as communicating with a public web service or safety-critical physical action cannot be truly rewound, i.e. firing a missile. Such operations, therefore, fall outside the theoretical scope of our approach and should be addressed through a controllable test environment.

Finally, while we have not applied our debugging recipe to other languages we believe our guidelines provide a clear path. As an example the same ideas could be applied in a vastly different environment, i.e. C in combination with GDB. First, all I/O primitives would have to be rewritten in a dedicated C library, making them easy to identify. Each I/O primitive would also need to be accompanied by a compensating action. GDB could then place breakpoints on each of these I/O primitives to intercept execution at the exact moments required for taking state snapshots. The snapshotting mechanism could be implemented with the memory read/write commands to save and later restore the program's state. While more involved, implementing the compensating actions is also feasible. GDB can use its memory write capability to directly manipulate the program's memory and stack, effectively executing the "undo" logic for an I/O operation. Our formal semantics provide the precise blueprint for this process, dictating exactly when to take snapshots and how to apply these compensating actions when stepping backwards.

6 Related Work

Our work builds directly on WebAssembly [32, 78, 79] and WARDuino [54], as we have discussed in Section 3.4 and Section 4. In this section, we present an overview of further related work.

Multiverse debuggers. Multiverse debugging has emerged as a powerful technique to debug non-deterministic program behavior, by allowing multiple execution paths to be explored simultaneously. It was proposed by Torres Lopez et al. [92] to debug actor-based programs with a prototype called Voyager [31], that worked directly on the operational semantics of the language defined in PLT Redex [21]. Several works have expanded on multiverse debugging; Pasquier et al. [72] introduced user-defined reduction rules to shrink the search space for multiverse-wide breakpoint lookup, and Pasquier et al. [71] introduced temporal breakpoints that allow users to reason about the future execution of a program using linear temporal logic. In contrast to MIO, existing multiverse debuggers do not consider I/O operations, or their effects on the external environment. Additionally, previous multiverse debuggers only work on a model of the program execution.

Multiverse analysis. The idea of exploring the multiverse of possibilities, is more widely known as multiverse analysis. Within statistical analysis, it is a method that considers all possible combinations

of datasets and analysis simultaneously [87]. Within software development, there are several frameworks for exploratory programming [44] that allow developers to interact with the multiverse of source code versions [89]. This way programmers can actively explore the behavior of a program by experimenting with different code. The approach has led to *programming notebooks* [45, 73], and dedicated *explore-first IDEs* [43, 89]. Explore-first IDEs, such as the original by Steinert et al. [89], allow exploration of different source code versions in parallel. While these editors consider the variations in the program code itself, multiverse debuggers focus on variations of program execution caused by non-deterministic behavior for a single code base. Combining the two techniques could lead to a powerful development environment, and represents interesting future work.

Exploring execution trees. Many automatic verification and other analysis tools also explore the execution tree of a program, such as software *model checkers* [28, 40], *symbolic execution* [5, 14, 46], and *concolic execution* [29, 63, 84]. These techniques are great at automatically detecting program faults, however, they rely on a precise description of the problem or program specification, often in the form of a formal model. This is in stark contrast with debuggers, which are tools to help developers find mistakes for which no precise formula exists, and for which the causes are often unknown. Despite the major differences, static analysis techniques could greatly help improve debuggers by providing the developers with more information. For multiverse debugging the techniques could help guide developers through large and complicated execution trees. Additionally, the techniques for handling the state explosion problem [41, 47, 93] developed for these analysis tools, can help reduce the number of redundant execution paths in multiverse debugging.

Reversible debuggers. Reversible debugging, also called back-in-time debugging, has existed for more than fifty years [6], and has been implemented with various strategies [19]. In spite of all the different implementations, few reversible debuggers reverse output effects. The most common approach is *record-replay debugging* [2, 12, 13, 20, 69, 76] that allows offline debugging with checkpoint-based traces. The recent RR framework [69] is the most advanced record-replay debugger to date. While replaying it does not reverse I/O operations, in fact, the operations are not performed at all, instead the external effects are recorded and replayed within the debugger. One of the earliest works, the Igor debugger [20], featured so-called *prestart routines*, which could perform certain actions after stepping back, such as updating the screen with the current frame buffer. This is one of the first attempts at dealing with external state, however, the solution was purely ad-hoc, and required significant user intervention. Additionally, dealing with I/O in a structured way through the prestart routines was still too costly at the time. There is also no proof of soundness, or any characterization of which prestart routines lead to correct debugging behavior. Another approach, *omniscient debugging* [57, 74], records the entire execution of a program, allowing free offline exploration of the entire history, and enabling advanced queries on causal relationships in the execution [74]. A third approach is based on *reversible programming languages* [26, 49, 51]. While not applicable in all scenarios, since it requires a fully reversible language, this approach can enable more advanced features, such as reversing only parts of a concurrent process, while still remaining consistent with the forwards execution [49]. The reversible LISP debugger by Lieberman [60] not only redraws the graphical output, but also links graphics with their responsible source code. Reversible debuggers for the *graphical programming language* Scratch [62], namely Blink [90] and NuzzleBug [18], also redraw the graphical output when stepping back. However, in all these debuggers, the output effects are internal to the system. For the Scratch debuggers, the visual output is actually part of the execution model [62].

Reversible programming languages. The concept of reversible computation has a longstanding history in computer science [8, 68, 97], with the most notable models for reversibility being reversible

Turing machines [4], and reversible circuits [80]. Furthermore, the design of reversible languages has evolved into its own field of study [27], with languages for most programming paradigms, such as the imperative, and first reversible language, Janus [48, 61, 95], several functional languages [66, 96], object-oriented languages [33, 34, 83], monadic computation [35], and languages for concurrent systems [17, 36, 81]. Several works have investigated how reversible languages can help reversible debuggers [15, 19, 51], however, full computational reversibility is not necessary for back-in-time debugging [19]. Moreover, these reversible languages do not consider output effects on the external world, with a few notable exceptions in the space of proprietary languages for industrial robots.

Reverse execution of industrial robots. While numerous examples can be imagined where actions affecting the environment cannot be easily reversed, there are sufficient scenario's where this is possible, for reverse execution to be widely used in industry. The reversible language by Schultz [82] is particularly interesting. The work proposes a system for error handling in robotics applications through reverse execution, and identifies two types of reversibility; direct and indirect. Through our compensating actions, MIO is able to handle both directly and indirectly reversible actions. Laursen et al. [52] propose a reversible domain-specific language for robotic assembly programs, SCP-RASQ. While we do not focus on a single specific application domain, this work does show how reversible output primitives are possible for advanced robotics applications. SCP-RASQ uses a similar system of user-defined compensating actions, to reverse indirectly reversible operations. Using these kinds of languages, we believe that the MIO debugger could be extended to support more complex output primitives, which could control industrial robots.

Reversibility. The concept of reversibility is well understood on a theoretical level, for both sequential context [56], and concurrent systems. The latter is much more complex, and has lead to two major definitions; causal-consistent reversibility [17, 50], and time reversibility [42, 94]. Causal-consistent reversibility is the idea that an action can only be reversed after all subsequent dependent actions have been reversed [50]. This ensures that all consequences of an action have been undone before reversing, and the system always returns to a past consistent state. On the other hand, time reversibility only considers the stochastic behavior when time is reversed [9, 42, 94]. However, it has recently been shown that causal-consistency implies time reversibility [9]. Our debugger works on a single-threaded language, where the non-determinism is introduced by the input operations. In our work, the undo actions are causally consistent in the single-threaded world. We believe that we can extend MIO to support concurrent languages, and that the existing literature [26, 51] can help to ensure it stays causally consistent.

Remote debugging on microcontrollers. In remote debugging [77], a debugger frontend is connected to a remote debugger backend running the program being debugged. The MIO debugger uses remote debugging to mitigate some limitations of microcontrollers, an approach that has been adopted for many embedded systems [75, 85, 86]. These debuggers fall in two categories; *stub* and *on-chip* [58]. A stub is a small piece of software that runs on the microcontroller, instrumenting the software being debugged, this is the approach taken in this work. On-chip debugging uses additional hardware to debug the embedded device, a common example are JTAG [1] hardware debuggers. These debuggers can interface with different software, such as the popular OpenOCD debugger [37]. Since the debugger front-end needs to communicate with the backend on the microcontroller it exacerbates the probe effect [25] and slows down the debugger. To address these limitations, a new form of remote debugging was proposed called out-of-place debugging [53, 64, 65]. This technique moves part of the debugging process to a more powerful machine, which can reduce debugging interference and speedup performance. The MIO debugger is already sufficiently fast, but a speed-up can likely be achieved by adopting out-of-place debugging.

Environment modeling. There are many environment interactions that can influence the possible input values and thereby the possible execution paths of a program. We have elided these interactions from the formal model and assume that I/O operations are independent, while our prototype does support defining simple *predictable dependencies* between I/O operations. Modeling the interactions between I/O operations is also hugely important for testing, and *environment modeling* has therefore been widely studied in this area [11]. Environment models are often used for automatic test generation [3, 16], and have also been applied to real-time embedded software [39].

Formalizing debuggers. Previous efforts to define debuggers formally have been incredibly varied in their depth and approach, and have not yet reached a consensus on any standard method. An early attempt used PowerEpsilon [99, 100] to define a denotational semantic describing the source mapping needed to debug a toy language [98]. In 2012, the work by Li and Li [59] focussed on automatic debuggers, and defined operational semantics for tracing, and for backwards searching based on those traces. In 1995, Bernstein and Stark [10] defined a debugger in terms of an underlying language semantic for the first time, an approach we adopted in this work as well. In fact, the approach is followed by a number of later works [22, 38, 54, 91], including the work by Torres Lopez et al. [92] in 2019, which defines the correctness of their debugger in terms of the non-interference with the underlying semantic. The correctness criteria requires each execution observed by either semantic is observed by the other, which is similar to our soundness and completeness theorems rolled into one. A more recent work presented a novel abstract debugger, that uses static analysis to allow developers to explore abstract program states rather than concrete ones [38]. The work defines operational semantics for their abstract debugger, and for a concrete debugger. The soundness of the abstract debugger is defined in terms this concrete debugger, where every debugging session in the concrete world is guaranteed to correspond to a session in the abstract world. The opposite direction cannot hold since the static analysis relies on an over-approximation, which means there can always be sessions in the abstract world which are impossible in the concrete world. This is in stark contrast with the soundness theorem in our work, which states that any path in the debugging semantics can be observed in the underlying language semantics.

7 Conclusion

While existing multiverse debuggers have shown promise in abstract settings, they struggled to adapt to concrete programming languages and I/O operations. In this article, we address these limitations by presenting a novel approach that seamlessly integrates multiverse debugging with a full-fledged WebAssembly virtual machine. This is the first implementation that enables multiverse debugging for microcontrollers. Our approach improves current multiverse debuggers by being able to provide multiverse debugging in the face of a set of well-defined I/O primitives. We have formalized our approach and give a soundness proof. We have implemented our approach and have given various examples showcasing how our approach can deal with a wide range of specialized I/O primitives, ranging from non-deterministic input sensors, to I/O pins and even steering motors. Our sparse snapshotting approach delivers reasonable performance even on a restricted microcontroller platform. Our initial implementation provides a substantial benefit over existing approaches, but we believe there are further opportunities to relax the constraints on I/O primitives further. For example, our current implementation only supports simple dependencies between I/O actions, but we believe this could be relaxed further by introducing an explicit rule language so that programmers can define more complex dependencies between the I/O actions.

Acknowledgements

We thank Francisco Ferreira Ruiz for providing the Open Bot Brain hardware, Jonas Sys for testing the GUI, the members of the VUB DisCo group for fruitful discussions, and all the contributors of the open source WARDuino VM. *Funding:* Tom Lauwaerts is funded by a project from the Research Foundation Flanders (FWO) [grant number FWOOPR2020008201].

Data-Availability Statement

The debugger technique proposed in this article is implemented and available as an open-source project (github.com/TOPLLab/MIO) developed on top of the open-source WARDuino virtual machine. The software is also available as research artifact archived and available on Zenodo [88].

References

[1] 2013. IEEE Standard for Test Access Port and Boundary-Scan Architecture. *IEEE Std 1149.1-2013 (Revision of IEEE Std 1149.1-2001)* (May 2013), 1–444. doi:10.1109/IEEESTD.2013.6515989

[2] H. Agrawal, R.A. De Millo, and E.H. Spafford. 1991. An Execution-Backtracking Approach to Debugging. *IEEE Software* 8, 3 (May 1991), 21–26. doi:10.1109/52.88940

[3] Mikhail Auguston, James Bret Michael, and Man-Tak Shing. 2005. Environment Behavior Models for Scenario Generation and Testing Automation. *SIGSOFT Softw. Eng. Notes* 30, 4 (May 2005), 1–6. doi:10.1145/1082983.1083284

[4] Holger Bock Axelsen and Robert Glück. 2016. On Reversible Turing Machines and Their Function Universality. *Acta Informatica* 53, 5 (Aug. 2016), 509–543. doi:10.1007/s00236-015-0253-y

[5] Roberto Baldoni, Emilio Coppa, Daniele Cono D'elia, Camil Demetrescu, and Irene Finocchi. 2018. A Survey of Symbolic Execution Techniques. *ACM Comput. Surv.* 51, 3 (May 2018), 50:1–50:39. doi:10.1145/3182657

[6] R. M. Balzer. 1969. EXDAMS: Extendable Debugging and Monitoring System. In *Proceedings of the May 14-16, 1969, Spring Joint Computer Conference (AFIPS '69 (Spring))*. Association for Computing Machinery, New York, NY, USA, 567–580. doi:10.1145/1476793.1476881

[7] R. Battagline. 2021. *The Art of WebAssembly: Build Secure, Portable, High-Performance Applications*. No Starch Press.

[8] Charles H. Bennett. 1988. Notes on the History of Reversible Computation. *IBM Journal of Research and Development* 32, 1 (Jan. 1988), 16–23. doi:10.1147/rd.321.0016

[9] Marco Bernardo, Ivan Lanese, Andrea Marin, Claudio A. Mezzina, Sabina Rossi, and Claudio Sacerdoti Coen. 2023. Causal Reversibility Implies Time Reversibility. In *Quantitative Evaluation of Systems*, Nils Jansen and Mirco Tribastone (Eds.). Springer Nature Switzerland, Cham, 270–287. doi:10.1007/978-3-031-43835-6_19

[10] Karen L. Bernstein and Eugene W. Stark. 1995. Operational Semantics of a Focusing Debugger. *Electronic Notes in Theoretical Computer Science* 1 (Jan. 1995), 13–31. doi:10.1016/S1571-0661(04)80002-1

[11] M.R. Blackburn. 1998. Using Models for Test Generation and Analysis. In *17th DASC. AIAA/IEEE/SAE. Digital Avionics Systems Conference. Proceedings (Cat. No. 98CH36267)*, Vol. 1. C45/1–C45/8 vol.1. doi:10.1109/DASC.1998.741501

[12] Bob Boothe. 2000. Efficient Algorithms for Bidirectional Debugging. In *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation (PLDI '00)*. Association for Computing Machinery, New York, NY, USA, 299–310. doi:10.1145/349299.349339

[13] Brian Burg, Richard Bailey, Amy J. Ko, and Michael D. Ernst. 2013. Interactive Record/Replay for Web Application Debugging. In *Proceedings of the 26th Annual ACM Symposium on User Interface Software and Technology (UIST '13)*. Association for Computing Machinery, New York, NY, USA, 473–484. doi:10.1145/2501988.2502050

[14] Cristian Cadar, Patrice Godefroid, Sarfraz Khurshid, Corina S. P ăsăreanu, Koushik Sen, Nikolai Tillmann, and Willem Visser. 2011. Symbolic Execution for Software Testing in Practice: Preliminary Assessment. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE '11)*. Association for Computing Machinery, New York, NY, USA, 1066–1071. doi:10.1145/1985793.1985995

[15] Shyh-Kwei Chen, W.K. Fuchs, and Jen-Yao Chung. 2001. Reversible Debugging Using Program Instrumentation. *IEEE Transactions on Software Engineering* 27, 8 (Aug. 2001), 715–727. doi:10.1109/32.940726

[16] S. R. Dalal, A. Jain, N. Karunanithi, J. M. Leaton, C. M. Lott, G. C. Patton, and B. M. Horowitz. 1999. Model-Based Testing in Practice. In *Proceedings of the 21st International Conference on Software Engineering (ICSE '99)*. Association for Computing Machinery, New York, NY, USA, 285–294. doi:10.1145/302405.302640

[17] Vincent Danos and Jean Krivine. 2004. Reversible Communicating Systems. In *CONCUR 2004 - Concurrency Theory*, Philippa Gardner and Nobuko Yoshida (Eds.). Springer, Berlin, Heidelberg, 292–307. doi:10.1007/978-3-540-28644-8_19

[18] Adina Deiner and Gordon Fraser. 2024. NuzzleBug: Debugging Block-Based Programs in Scratch. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. ACM, Lisbon, Portugal, 1–13. doi:10.1145/3597503.

3623331
[19] Jakob Engblom. 2012. A Review of Reverse Debugging. In *Proceedings of the 2012 System, Software, SoC and Silicon Debug Conference*. 1–6.
[20] Stuart I. Feldman and Channing B. Brown. 1988. IGOR: A System for Program Debugging via Reversible Execution. *ACM SIGPLAN Notices* 24, 1 (Nov. 1988), 112–123. doi:10.1145/69215.69226
[21] Matthias Felleisen, Robert Bruce Findler, and Matthew Flatt. 2009. *Semantics Engineering with PLT Redex*. Mit Press.
[22] GianLuigi Ferrari and Emilio Tuosto. 2001. A Debugging Calculus for Mobile Ambients. In *Proceedings of the 2001 ACM Symposium on Applied Computing*. ACM, Las Vegas Nevada USA. doi:10.1145/372202.380701
[23] Francisco Ferreira Ruiz, Ben Collins, and et. al. 2024. Open Bot Brain.
[24] Flavio Frattini, Roberto Pietrantuono, and Stefano Russo. 2016. Reproducibility of Software Bugs. In *Principles of Performance and Reliability Modeling and Evaluation: Essays in Honor of Kishor Trivedi on His 70th Birthday*, Lance Fiondella and Antonio Puliafito (Eds.). Springer International Publishing, Cham, 551–565. doi:10.1007/978-3-319-30599-8_21
[25] Jason Gait. 1986. A Probe Effect in Concurrent Programs. *Software: Practice and Experience* 16, 3 (1986), 225–233. doi:10.1002/spe.4380160304
[26] Elena Giachino, Ivan Lanese, and Claudio Antares Mezzina. 2014. Causal-Consistent Reversible Debugging. In *Fundamental Approaches to Software Engineering (Lecture Notes in Computer Science)*, Stefania Gnesi and Arend Rensink (Eds.). Springer, Berlin, Heidelberg, 370–384. doi:10.1007/978-3-642-54804-8_26
[27] Robert Glück and Tetsuo Yokoyama. 2023. Reversible Computing from a Programming Language Perspective. *Theoretical Computer Science* 953 (April 2023), 113429. doi:10.1016/j.tcs.2022.06.010
[28] Patrice Godefroid. 1997. Model Checking for Programming Languages Using VeriSoft. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '97)*. Association for Computing Machinery, New York, NY, USA, 174–186. doi:10.1145/263699.263717
[29] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: Directed Automated Random Testing. *SIGPLAN Not.* 40, 6 (June 2005), 213–223. doi:10.1145/1064978.1065036
[30] Robbert Gurdeep Singh. 2022. *Taming Nondeterminism : Programming Language Abstractions and Tools for Dealing with Nondeterministic Programs*. Ph. D. Dissertation. Ghent University.
[31] Robbert Gurdeep Singh, Carmen Torres Lopez, Stefan Marr, Elisa Gonzalez Boix, and Christophe Scholliers. 2019. Multiverse Debugging: Non-Deterministic Debugging for Non-Deterministic Programs (Artifact). *Dagstuhl Artifacts Series* 5, 2 (2019), 4:1–4:3. doi:10.4230/DARTS.5.2.4
[32] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. 2017. Bringing the Web up to Speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017)*. Association for Computing Machinery, New York, NY, USA, 185–200. doi:10.1145/3062341.3062363
[33] Tue Haulund, Torben Ægidius Mogensen, and Robert Glück. 2017. Implementing Reversible Object-Oriented Language Features on Reversible Machines. In *Reversible Computation*, Iain Phillips and Hafizur Rahaman (Eds.). Springer International Publishing, Cham, 66–73. doi:10.1007/978-3-319-59936-6_5
[34] Lasse Hay-Schmidt, Robert Glück, Martin Holm Cservenka, and Tue Haulund. 2021. Towards a Unified Language Architecture for Reversible Object-Oriented Programming. In *Reversible Computation*, Shigeru Yamashita and Tetsuo Yokoyama (Eds.). Springer International Publishing, Cham, 96–106. doi:10.1007/978-3-030-79837-6_6
[35] Chris Heunen and Martti Karvonen. 2015. Reversible Monadic Computing. *Electronic Notes in Theoretical Computer Science* 319 (Dec. 2015), 217–237. doi:10.1016/j.entcs.2015.12.014
[36] James Hoey, Irek Ulidowski, and Shoji Yuen. 2018. Reversing Parallel Programs with Blocks and Procedures. doi:10.48550/arXiv.1808.08651
[37] Hubert Högl and Dominic Rath. 2006. Open On-Chip Debugger–Openocd–. *Fakultat fur Informatik, Tech. Rep* (2006).
[38] Karoliine Holter, Juhan Oskar Hennoste, Patrick Lam, Simmo Saan, and Vesal Vojdani. 2024. Abstract Debuggers: Exploring Program Behaviors Using Static Analysis Results. In *Proceedings of the 2024 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! '24)*. Association for Computing Machinery, New York, NY, USA, 130–146. doi:10.1145/3689492.3690053
[39] Muhammad Zohaib Iqbal, Andrea Arcuri, and Lionel Briand. 2015. Environment Modeling and Simulation for Automated Testing of Soft Real-Time Embedded Software. *Software & Systems Modeling* 14, 1 (Feb. 2015), 483–524. doi:10.1007/s10270-013-0328-6
[40] Ranjit Jhala and Rupak Majumdar. 2009. Software Model Checking. *ACM Comput. Surv.* 41, 4 (Oct. 2009), 21:1–21:54. doi:10.1145/1592434.1592438
[41] Vineet Kahlon, Chao Wang, and Aarti Gupta. 2009. Monotonic Partial Order Reduction: An Optimal Symbolic Partial Order Reduction Technique. In *Computer Aided Verification*, Ahmed Bouajjani and Oded Maler (Eds.). Springer, Berlin, Heidelberg, 398–413. doi:10.1007/978-3-642-02658-4_31

[42] F.P. Kelly. 1981. Reversibility and Stochastic Networks / F.P. Kelly. *SERBIULA (sistema Librum 2.0)* 76 (June 1981). doi:10.2307/2287860

[43] Mary Beth Kery, Amber Horvath, and Brad Myers. 2017. Variolite: Supporting Exploratory Programming by Data Scientists. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems (CHI '17)*. Association for Computing Machinery, New York, NY, USA, 1265–1276. doi:10.1145/3025453.3025626

[44] Mary Beth Kery and Brad A. Myers. 2017. Exploring Exploratory Programming. In *2017 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 25–29. doi:10.1109/VLHCC.2017.8103446

[45] Mary Beth Kery, Marissa Radensky, Mahima Arya, Bonnie E. John, and Brad A. Myers. 2018. The Story in the Notebook: Exploratory Data Science Using a Literate Programming Tool. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems (CHI '18)*. Association for Computing Machinery, New York, NY, USA, 1–11. doi:10.1145/3173574.3173748

[46] James C. King. 1976. Symbolic Execution and Program Testing. *Commun. ACM* 19, 7 (July 1976), 385–394. doi:10.1145/360248.360252

[47] R. Kurshan, V. Levin, M. Minea, D. Peled, and H. Yenigün. 1998. Static Partial Order Reduction. In *Tools and Algorithms for the Construction and Analysis of Systems*, Bernhard Steffen (Ed.). Springer, Berlin, Heidelberg, 345–357. doi:10.1007/BFb0054182

[48] Pietro Lami, Ivan Lanese, and Jean-Bernard Stefani. 2024. A Small-Step Semantics for Janus. In *Reversible Computation*, Torben Ægidius Mogensen and Łukasz Mikulski (Eds.). Springer Nature Switzerland, Cham, 105–123. doi:10.1007/978-3-031-62076-8_8

[49] Ivan Lanese. 2018. From Reversible Semantics to Reversible Debugging. In *Reversible Computation (Lecture Notes in Computer Science)*, Jarkko Kari and Irek Ulidowski (Eds.). Springer International Publishing, Cham, 34–46. doi:10.1007/978-3-319-99498-7_2

[50] Ivan Lanese, Claudio Mezzina, and Francesco Tiezzi. 2014. Causal-Consistent Reversibility. (Nov. 2014).

[51] Ivan Lanese, Naoki Nishida, Adrián Palacios, and Germán Vidal. 2018. CauDEr: A Causal-Consistent Reversible Debugger for Erlang. In *Functional and Logic Programming (Lecture Notes in Computer Science)*, John P. Gallagher and Martin Sulzmann (Eds.). Springer International Publishing, Cham, 247–263. doi:10.1007/978-3-319-90686-7_16

[52] Johan Sund Laursen, Lars-Peter Ellekilde, and Ulrik Pagh Schultz. 2018. Modelling Reversible Execution of Robotic Assembly. *Robotica* 36, 5 (May 2018), 625–654. doi:10.1017/S0263574717000613

[53] Tom Lauwaerts, Carlos Rojas Castillo, Robbert Gurdeep Singh, Matteo Marra, Christophe Scholliers, and Elisa Gonzalez Boix. 2022. Event-Based Out-of-Place Debugging. In *Proceedings of the 19th International Conference on Managed Programming Languages and Runtimes (MPLR '22)*. Association for Computing Machinery, New York, NY, USA, 85–97. doi:10.1145/3546918.3546920

[54] Tom Lauwaerts, Robbert Gurdeep Singh, and Christophe Scholliers. 2024. WARDuino: An Embedded WebAssembly Virtual Machine. *Journal of Computer Languages* (Feb. 2024), 101268. doi:10.1016/j.cola.2024.101268

[55] Tom Lauwaerts, Maarten Steevens, and Christophe Scholliers. 2025. MIO: Multiverse Debugging in the Face of Input/Output – Extended Version with Additional Appendices. (Sept. 2025). doi:10.48550/arXiv.2509.06845

[56] George B. Leeman. 1986. A Formal Approach to Undo Operations in Programming Languages. *ACM Transactions on Programming Languages and Systems* 8, 1 (Jan. 1986), 50–87. doi:10.1145/5001.5005

[57] Bil Lewis. 2003. Debugging Backwards in Time. doi:10.48550/arXiv.cs/0310016

[58] Hongwei Li, Yaping Xu, Fangsheng Wu, and Changhong Yin. 2009. Research of "Stub" Remote Debugging Technique. In *2009 4th International Conference on Computer Science & Education*. 990–994. doi:10.1109/ICCSE.2009.5228140

[59] Wei Li and Ning Li. 2012. A Formal Semantics for Program Debugging. *Science China Information Sciences* 55, 1 (Jan. 2012), 133–148. doi:10.1007/s11432-011-4530-2

[60] Henry Lieberman. 1997. ZStep 95: A Reversible, Animated Source Code Stepper. *Software Visualization: Programming as a Multimedia Experience* (1997).

[61] Christopher Lutz and Howard Derby. 1986. Janus: A Time-Reversible Language. *Letter to R. Landauer* 2 (1986).

[62] John Maloney, Mitchel Resnick, Natalie Rusk, Brian Silverman, and Evelyn Eastmond. 2010. The Scratch Programming Language and Environment. *ACM Transactions on Computing Education* 10, 4 (Nov. 2010), 1–15. doi:10.1145/1868358.1868363

[63] Filipe Marques, José Fragoso Santos, Nuno Santos, and Pedro Adão. 2022. Concolic Execution for WebAssembly. In *36th European Conference on Object-Oriented Programming (ECOOP 2022) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 222)*, Karim Ali and Jan Vitek (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 11:1–11:29. doi:10.4230/LIPIcs.ECOOP.2022.11

[64] Matteo Marra, Guillermo Polito, and Elisa Gonzalez Boix. 2021. Practical Online Debugging of Spark-like Applications. In *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*. 620–631. doi:10.1109/QRS54544.2021.00072

[65] Matteo Marra, Guillermo Polito, and Elisa Gonzalez Boix. 2018. Out-Of-Place Debugging: A Debugging Architecture to Reduce Debugging Interference. *The Art, Science, and Engineering of Programming* 3, 2 (Nov. 2018), 3:1–3:29. doi:10.22152/programming-journal.org/2019/3/3

[66] Kazutaka Matsuda and Meng Wang. 2020. Sparcl: A Language for Partially-Invertible Computation. *Proceedings of the ACM on Programming Languages* 4, ICFP (Aug. 2020), 118:1–118:31. doi:10.1145/3409000

[67] Charles E. McDowell and David P. Helmbold. 1989. Debugging Concurrent Programs. *ACM Comput. Surv.* 21, 4 (Dec. 1989), 593–622. doi:10.1145/76894.76897

[68] Claudio Antares Mezzina, Rudolf Schlatte, Robert Glück, Tue Haulund, James Hoey, Martin Holm Cservenka, Ivan Lanese, Torben Æ. Mogensen, Harun Siljak, Ulrik P. Schultz, and Irek Ulidowski. 2020. Software and Reversible Systems: A Survey of Recent Activities. In *Reversible Computation: Extending Horizons of Computing: Selected Results of the COST Action IC1405*, Irek Ulidowski, Ivan Lanese, Ulrik Pagh Schultz, and Carla Ferreira (Eds.). Springer International Publishing, Cham, 41–59. doi:10.1007/978-3-030-47361-7_2

[69] Robert O'Callahan, Chris Jones, Nathan Froyd, Kyle Huey, Albert Noll, and Nimrod Partush. 2017. Engineering Record and Replay for Deployability. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. 377–389.

[70] Matthias Pasquier, Ciprian Teodorov, Frédé ric Jouault, Matthias Brun, and Loïc Lagadec. 2023. Debugging Paxos in the UML Multiverse. In *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. 811–820. doi:10.1109/MODELS-C59198.2023.00130

[71] Matthias Pasquier, Ciprian Teodorov, Frédé ric Jouault, Matthias Brun, Luka Le Roux, and Loïc Lagadec. 2023. Temporal Breakpoints for Multiverse Debugging. In *Proceedings of the 16th ACM SIGPLAN International Conference on Software Language Engineering (SLE 2023)*. Association for Computing Machinery, New York, NY, USA, 125–137. doi:10.1145/3623476.3623526

[72] Matthias Pasquier, Ciprian Teodorov, Frédé ric Jouault, Matthias Brun, Luka Le Roux, and Loïc Lagadec. 2022. Practical Multiverse Debugging through User-Defined Reductions: Application to UML Models. In *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems (MODELS '22)*. Association for Computing Machinery, New York, NY, USA, 87–97. doi:10.1145/3550355.3552447

[73] Fernando Perez and Brian E. Granger. 2007. IPython: A System for Interactive Scientific Computing. *Computing in Science & Engineering* 9, 3 (May 2007), 21–29. doi:10.1109/MCSE.2007.53

[74] Guillaume Pothier, Éric Tanter, and José Piquer. 2007. Scalable Omniscient Debugging. *SIGPLAN Not.* 42, 10 (Oct. 2007), 535–552. doi:10.1145/1297105.1297067

[75] Albert Pötsch, Florian Haslhofer, and Andreas Springer. 2017. Advanced Remote Debugging of LoRa-enabled IoT Sensor Nodes. In *Proceedings of the Seventh International Conference on the Internet of Things (IoT '17)*. Association for Computing Machinery, New York, NY, USA, 1–2. doi:10.1145/3131542.3140259

[76] Michiel Ronsse and Koen De Bosschere. 1999. RecPlay: A Fully Integrated Practical Record/Replay System. *ACM Transactions on Computer Systems* 17, 2 (May 1999), 133–152. doi:10.1145/312203.312214

[77] Jonathan B. Rosenberg. 1996. *How Debuggers Work: Algorithms, Data Structures, and Architecture*. John Wiley & Sons, Inc., USA.

[78] Andreas Rossberg. 2019. WebAssembly (Release 1.0). https://webassembly.github.io/spec/.

[79] Andreas Rossberg. 2023. WebAssembly (Release 2.0). https://webassembly.github.io/spec/.

[80] Mehdi Saeedi and Igor L. Markov. 2013. Synthesis and Optimization of Reversible Circuits—a Survey. *ACM Comput. Surv.* 45, 2 (March 2013), 21:1–21:34. doi:10.1145/2431211.2431220

[81] Markus Schordan, Tomas Oppelstrup, David Jefferson, Peter D. Barnes, and Dan Quinlan. 2016. Automatic Generation of Reversible C++ Code and Its Performance in a Scalable Kinetic Monte-Carlo Application. In *Proceedings of the 2016 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (SIGSIM-PADS '16)*. Association for Computing Machinery, New York, NY, USA, 111–122. doi:10.1145/2901378.2901394

[82] Ulrik Pagh Schultz. 2020. Reversible Control of Robots. In *Reversible Computation: Extending Horizons of Computing: Selected Results of the COST Action IC1405*, Irek Ulidowski, Ivan Lanese, Ulrik Pagh Schultz, and Carla Ferreira (Eds.). Springer International Publishing, Cham, 177–186. doi:10.1007/978-3-030-47361-7_8

[83] Ulrik Pagh Schultz and Holger Bock Axelsen. 2016. Elements of a Reversible Object-Oriented Language. In *Reversible Computation*, Simon Devitt and Ivan Lanese (Eds.). Springer International Publishing, Cham, 153–159. doi:10.1007/978-3-319-40578-0_10

[84] Koushik Sen and Gul Agha. 2006. Automated Systematic Testing of Open Distributed Programs. In *Fundamental Approaches to Software Engineering*, Luciano Baresi and Reiko Heckel (Eds.). Springer, Berlin, Heidelberg, 339–356. doi:10.1007/11693017_25

[85] Gašper Skvarč Božič, Ibai Irigoyen Ceberio, and Albrecht Mayer. 2024. In-Field Debugging of Automotive Microcontrollers for Highest System Availability. In *Proceedings of the 2nd ACM International Workshop on Future Debugging Techniques (DEBT 2024)*. Association for Computing Machinery, New York, NY, USA, 2–8. doi:10.1145/3678720.3685314

[86] Karl Söderby and Ubi De Feo. 2024. Debugging with the Arduino IDE 2.0. https://docs.arduino.cc/software/ide-v2/tutorials/ide-v2-debugger.
[87] Sara Steegen, Francis Tuerlinckx, Andrew Gelman, and Wolf Vanpaemel. 2016. Increasing Transparency Through a Multiverse Analysis. *Perspectives on Psychological Science* 11, 5 (Sept. 2016), 702–712. doi:10.1177/1745691616658637
[88] Maarten Steevens, Tom Lauwaerts, and Christophe Scholliers. 2025. *TOPLLab/MIO: 0.1.4.* doi:10.5281/zenodo.15838624
[89] Bastian Steinert, Damien Cassou, and Robert Hirschfeld. 2012. CoExist: Overcoming Aversion to Change. *SIGPLAN Not.* 48, 2 (Oct. 2012), 107–118. doi:10.1145/2480360.2384591
[90] Niko Strijbol, Robbe De Proft, Klaas Goethals, Bart Mesuere, Peter Dawyndt, and Christophe Scholliers. 2024. Blink: An Educational Software Debugger for Scratch. *SoftwareX* 25 (Feb. 2024), 101617. doi:10.1016/j.softx.2023.101617
[91] Carmen Torres Lopez, Elisa Gonzalez Boix, Christophe Scholliers, Stefan Marr, and Hanspeter Mössenböck. 2017. A Principled Approach towards Debugging Communicating Event-Loops. In *Proceedings of the 7th ACM SIGPLAN International Workshop on Programming Based on Actors, Agents, and Decentralized Control (AGERE 2017)*. Association for Computing Machinery, New York, NY, USA, 41–49. doi:10.1145/3141834.3141839
[92] Carmen Torres Lopez, Robbert Gurdeep Singh, Stefan Marr, Elisa Gonzalez Boix, and Christophe Scholliers. 2019. Multiverse Debugging: Non-Deterministic Debugging for Non-Deterministic Programs (Brave New Idea Paper). In *DROPS-IDN/v2/Document/10.4230/LIPIcs.ECOOP.2019.27*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.ECOOP.2019.27
[93] Antti Valmari. 1998. The State Explosion Problem. In *Lectures on Petri Nets I: Basic Models: Advances in Petri Nets*, Wolfgang Reisig and Grzegorz Rozenberg (Eds.). Springer, Berlin, Heidelberg, 429–528. doi:10.1007/3-540-65306-6_21
[94] Gideon Weiss. 1975. Time-Reversibility of Linear Stochastic Processes. *Journal of Applied Probability* 12, 4 (Dec. 1975), 831–836. doi:10.2307/3212735
[95] Tetsuo Yokoyama, Holger Bock Axelsen, and Robert Glück. 2008. Principles of a Reversible Programming Language. In *Proceedings of the 5th Conference on Computing Frontiers (CF '08)*. Association for Computing Machinery, New York, NY, USA, 43–54. doi:10.1145/1366230.1366239
[96] Tetsuo Yokoyama, Holger Bock Axelsen, and Robert Glück. 2012. Towards a Reversible Functional Language. In *Reversible Computation*, Alexis De Vos and Robert Wille (Eds.). Springer, Berlin, Heidelberg, 14–29. doi:10.1007/978-3-642-29517-1_2
[97] M. V. Zelkowitz. 1973. Reversible Execution. *Commun. ACM* 16, 9 (Sept. 1973), 566. doi:10.1145/362342.362360
[98] Ming-Yuan Zhu. 2001. Formal Specifications of Debuggers. *SIGPLAN Not.* 36, 9 (Sept. 2001), 54–63. doi:10.1145/609769.609778
[99] M.-Y. Zhu and C.-W. Wang. 1992. Program Derivation in PowerEpsilon. In *1992 Proceedings. The Sixteenth Annual International Computer Software and Applications Conference*. IEEE Computer Society, 206,207,208,209,210,211–206,207,208,209,210,211. doi:10.1109/CMPSAC.1992.217567
[100] Ming-Yuan Zhu and CW Wang. 1991. *A Higher-Order Lambda Calculus: PowerEpsilon.* Technical Report. Technical report, Beijing Institute of Systems Engineering, Beijing.

Received 2025-03-25; accepted 2025-08-12